

Introduction to safety-related software engineering

Focus on multitasking application design and implementation

Frédéric Thomas - fthomas.fr@gmail.com - Master

© Copyright 2015-2025, Frédéric Thomas. Creative Commons BY-SA 3.0 license. Latest update: March 16, 2025.

1

Summary

Chapter 1:
Engineering and
process overview

Chapter 2: Design
approches
overview

Chapter 3:
Implementation
concepts overview

Appendix

Practical works

*Which are the
engineering issues
and constraints?*

*Can you illustrate some
design approaches and
languages?*

*Can you illustrate
implementation concepts
and APIs?*

*Just for
advanced topics !*

2

Disclaimers

This support attempts to give an overview of issues, concepts and processes to design and develop a safety-related multitasking software with a focus on implementation for the last part. It doesn't focus on safety/cybersecurity analyses and verifications (tests) which are also important parts of a critical development process.

3

Engineering and process overview

Which are the engineering issues, constraints and good practices ?

Summary

1. Issues overview
2. Process overview for safety software (Software assurance)
3. Illustration with the DO-178C/ED-12C for aviation softwares
4. Good practices for requirements and designs



4

Safety-critical system

IEEE defines safety-critical software as:

Software who use in a system can result in unacceptable risk. Safety-critical software includes:

- software whose operation or failure to operate can lead to hazardous states
- software intended to recover from hazardous states
- software intended to mitigate the severity of an accident.

5

Boeing 737 Max System Architecture Example (1/4)



Fig. 1. New Leap Engine size on 737 MAX-8 (Right) vs. old engine on the 737 NG (Left) [4].



Fig. 3. AOA sensors location on the plane [1]

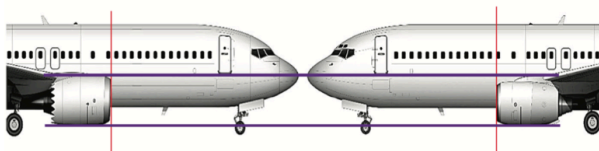


Fig. 2. New Leap Engine mount up and forward on 737 MAX-8 (Right) vs. 737 NG (Left) [3].

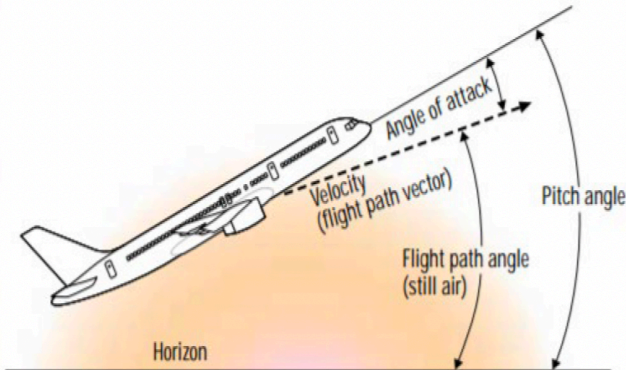
<https://www.american-cse.org/csce2023-ieee/pdfs/CSCE2023-5LpKs7cpb4k2UysbLCuOx/275900c664/275900c664.pdf>

<https://democrats-transportation.house.gov/committee-activity/boeing-737-max-investigation>

6

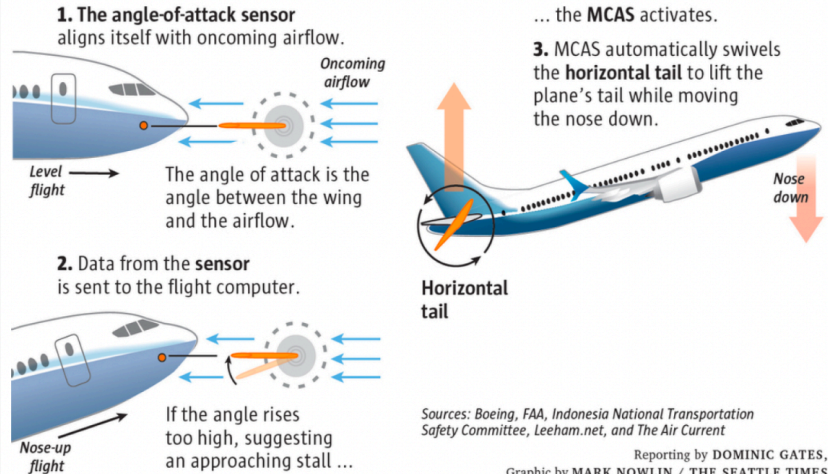
Boeing 737 Max System Architecture Example (2/4)

AOA, FLIGHT PATH ANGLE, AND PITCH ANGLE



Source: "What is Angle of Attack?" *AERO Magazine*, The Boeing Company, October 2000.

How the MCAS (Maneuvering Characteristics Augmentation System) works on the 737 MAX



Source: Dominic Gates and Mike Baker, "The inside story of MCAS: How Boeing's 737 MAX system gained power and lost safeguards," *The Seattle Times*, June 22, 2019 (Updated: June 24, 2019).

<https://www.american-cse.org/csce2023-ieee/pdfs/CSCE2023-5LpKs7cpb4k2UysbLCuOx/275900c664/275900c664.pdf>

<https://democrats-transportation.house.gov/committee-activity/boeing-737-max-investigation>

7

Boeing 737 Max System Architecture Example (3/4)

November 1, 2012 A Boeing test pilot takes more than 10 seconds to respond to uncommanded MCAS activation in a flight simulator and finds the condition "catastrophic." Boeing repeatedly referred to that test data in internal documents but never relayed that information to the FAA or to MAX pilots because there was no specific requirement to share it.	November 8, 2012 A Boeing Preliminary Design Decision Memo shows plans for an MCAS annunciator on the flight deck of the 737 MAX to indicate if MCAS fails to activate. However, the annunciator was not ultimately included on the aircraft.	May 4, 2013 Michael Teal, former Chief Project Engineer on the 737 MAX program, sends an email to senior Boeing managers indicating "concerns" about the addition of MCAS to the flight controls system and its impact on Boeing's ability to obtain Level B (non-simulator) pilot differences training for 737 MAX pilots.
May-June 2013 Boeing employees devised a strategy on June 7, 2013, to treat MCAS as an "addition to Speed Trim" to help prevent increased "cost" due to changed manuals. The strategy, approved by a Boeing Authorized Representative (AR), is outlined in an email, saying: "If we emphasize MCAS is a new function there may be greater certification and training impact."	July 8, 2015 Boeing notes its test pilot's slow, "catastrophic" reaction time to uncommanded MCAS activation in its Coordination Sheet for the first time, saying, "A typical reaction time was observed to be approximately 4 seconds. A slow reaction time scenario (> 10 seconds) found the failure to be catastrophic due to the inability to arrest the airplane overspeed." Boeing updated this record, citing the same information, six times from 2015 to 2018 but never shared this data with the FAA.	December 17, 2015 A Boeing AR asked in an email, "Are we vulnerable to single AOA sensor failures with the MCAS implementation or is there some checking that occurs?" In the end, MCAS was certified with a single AOA sensor and erroneous AOA data contributed to both 737 MAX accidents. Boeing is now implementing changes in the wake of both MAX crashes so that MCAS relies on two AOA sensors.
March 30, 2016 (8:00 am) Keith Levelsohn, Boeing's former 737 MAX program General Manager, and Michael Teal, former Chief Project Engineer on the MAX program, approve the redesign of MCAS that enables it to activate at lower speeds.	March 30, 2016 (11:16 am) Mark Forkner, former 737 Chief Technical Pilot, emails an FAA official in the Seattle AEG at 11:16 a.m. requesting permission to remove references to MCAS from the MAX's Flight Crew Operations Manual and training material. Unaware of the MCAS redesign, the FAA official grants this request. The Committee has been unable to determine if Mr. Forkner was aware that Boeing had approved a design change to MCAS at the time he emailed FAA.	June 15, 2016 A Boeing AR Advisor emailed a colleague and asked, "What happens when we have faulty AOA or Mach number?" The colleague responds, "As for faulty AOA and/or Mach number... if they are faulty then MCAS shuts down immediately." Faulty AOA data was a major contributing factor in both MAX crashes and MCAS did not shut down in either of those accidents.
June 16, 2016 Refusing to a Boeing test pilot's problem trimming the MAX due to repetitive MCAS activations, a Boeing engineer asks, "Is this considered a safety or certification issue?" On June 20, 2016, a colleague responds, "I don't think this is safety, other than [sic] the pilot could fight the MCAS input and over time find themselves in a huge mission." This is exactly what happened on both MAX aircraft that crashed.	October 29, 2018 Lion Air flight 610 crashes 13 minutes after takeoff killing 189 people. MCAS is quickly determined to be a major factor in the crash. Faulty AOA data erroneously caused MCAS to repetitively activate more than 20 times as the pilots battled MCAS for control of the aircraft. These safety issues were identified by Boeing's engineers years before the Lion Air crash, but they were not appropriately addressed or diminished.	March 10, 2019 Ethiopian Airlines flight 302 crashes six minutes after takeoff killing all 157 passengers and crew. The FAA is the last civil aviation authority to ground the 737 MAX on March 13, 2020, three days after the Ethiopian crash. The 737 MAX aircraft has been grounded worldwide ever since.

<https://www.american-cse.org/csce2023-ieee/pdfs/CSCE2023-5LpKs7cpb4k2UysbLCuOx/275900c664/275900c664.pdf>

<https://democrats-transportation.house.gov/committee-activity/boeing-737-max-investigation>

8

Boeing 737 Max System Architecture Example (4/4)



9

Safety-critical Engineering

c. Différences entre sûreté et sécurité

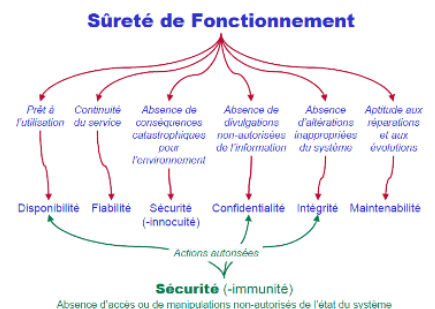
- Safety:

- Availability: readiness for correct service
- Reliability: continuity of correct service.
- Safety: absence of catastrophic consequences on the user(s) and the environment.
- Integrity: absence of improper system alterations.
- Maintainability: ability to undergo modifications and repairs

Sûreté : ensemble de mécanismes mis en place pour assurer la continuité de fonctionnement du système dans les conditions requises.

Sécurité : ensemble de mécanismes destinés à protéger l'information des utilisateurs ou processus n'ayant pas l'autorisation de la manipuler et d'assurer les accès autorisés.

Le périmètre de chacune des 2 notions n'est pas si clairement délimité dans la réalité : dans le cas de la voiture connectée on cherchera la sécurité et la sûreté.

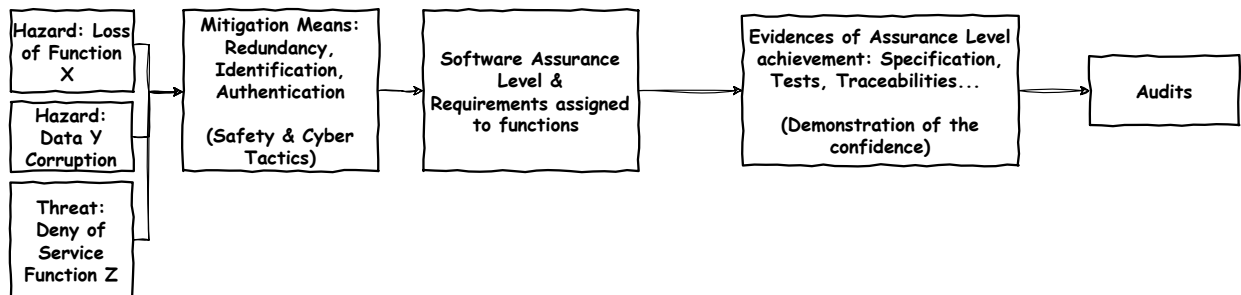
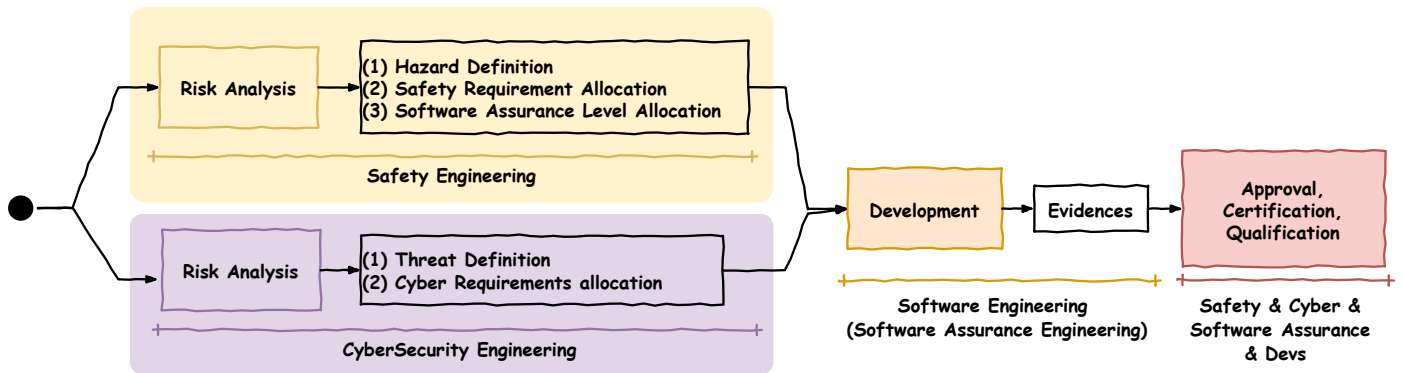


On constate sur le schéma que la notion de sécurité diffère selon le contexte :

- sécurité ► innocuité
- sécurité ► immunité

10

Safety-critical Engineering



11

Safety-critical Engineering

Tableau 7.3. — Tableau AMDE utilisé à Electricité de France

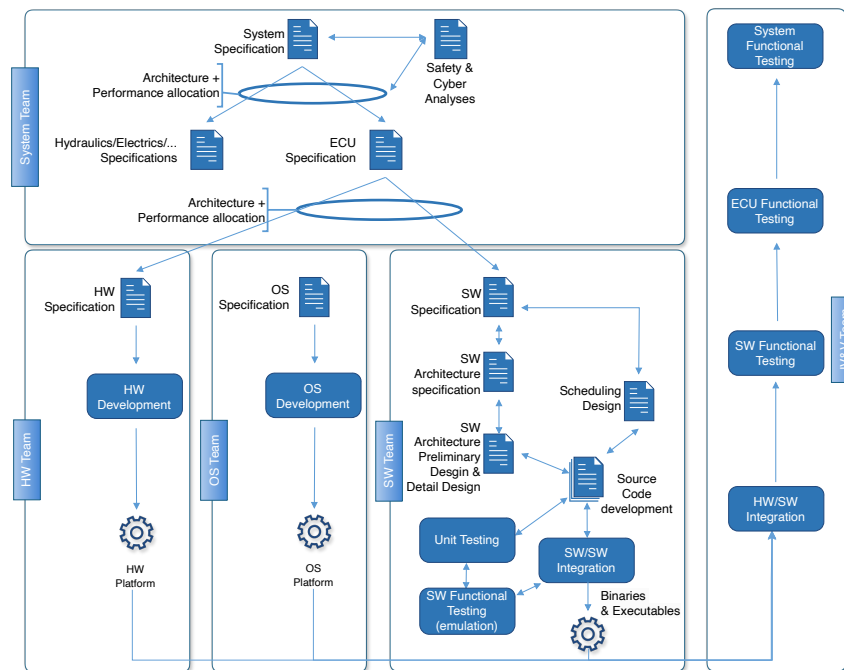
ANALYSE DES MODES DE DÉFAILLANCES DES COMPOSANTS ET DE LEURS EFFETS SUR LE SYSTÈME

PROJET :
SYSTÈME :

DOCUMENTS DE RÉFÉRENCE :

Identification du composant (Repères, désignation, type, lieu)	Fonctions, états	Modes de défaillance	Causes possibles d'une défaillance (Causes : internes, externes)	Effets sur le système	Effets sur les systèmes externes	Moyens de détection	Fréquence des inspections ou essais	Observations

Concrete System Engineering for safety software



13

Engineering issue: Confidence (1/3)

- This confidence is built on: determinism for the **features** and for the **tests**
 1. System behavior is always unique and invariant
 2. A causal chain between cause and the consequence effect can be established without a doubt. **The execution path is known and the same from one execution to another.**
 3. Different implementations (example: Different implementations of the scheduling) have identical results (except for the sizing)
 4. Test scenarios are predictable and their results reproducible
 5. The consequence of failures shall be deterministic
 6. Faults are confined, detected and their impacts are mastered (degraded mode)
 7. Non functional properties are mastered (performances)

14



Engineering issue: Confidence (2/3)

- This confidence is built on:
 1. System and Software engineering
 2. Safety and Cybersecurity by design
 3. No embedded behaviors that humans can not monitor and control
 4. Spatial and timing partitioning
 5. Fault tolerant approach: Assist to built correct & safe application
 6. Mastering execution: Safe & efficient resource sharing
 7. Diversification by design
 8. Relevant verification & validation: Ensure behavioral determinism
 9. In-depth defense
 10. Requirements, tests and codes are traced
 11. Development methods are defined and applied
 12. The person verifying the item (such as a requirement or source code or test) may not be the person who verifies/validates/approuves the item and this separation must be clearly documented

15

Engineering issue: Confidence (3/3)

- This confidence is built on processes reliability and processes management: quality assurance with a legal approval process (Certification)
 1. Industrial systems: Cenelec IEC 61508 (Safety integrity level (SIL): SIL1, SIL2, SIL3, SIL4 (most critical))
 2. Airborne systems and equipment: DO-178C/ED-12C (Design Assurance Level (DAL): E, D, C, B, A (most critical))
 3. Communication, Navigation, Surveillance and Air Traffic Management Systems: DO-278/ED-109A (Assurance Level (AL): 6, 5, 4, 3, 2, 1 (most critical))
 4. Automotive: ISO 26262 (Automotive Safety Integrity Level (ASIL): A, B, C, D (most critical))

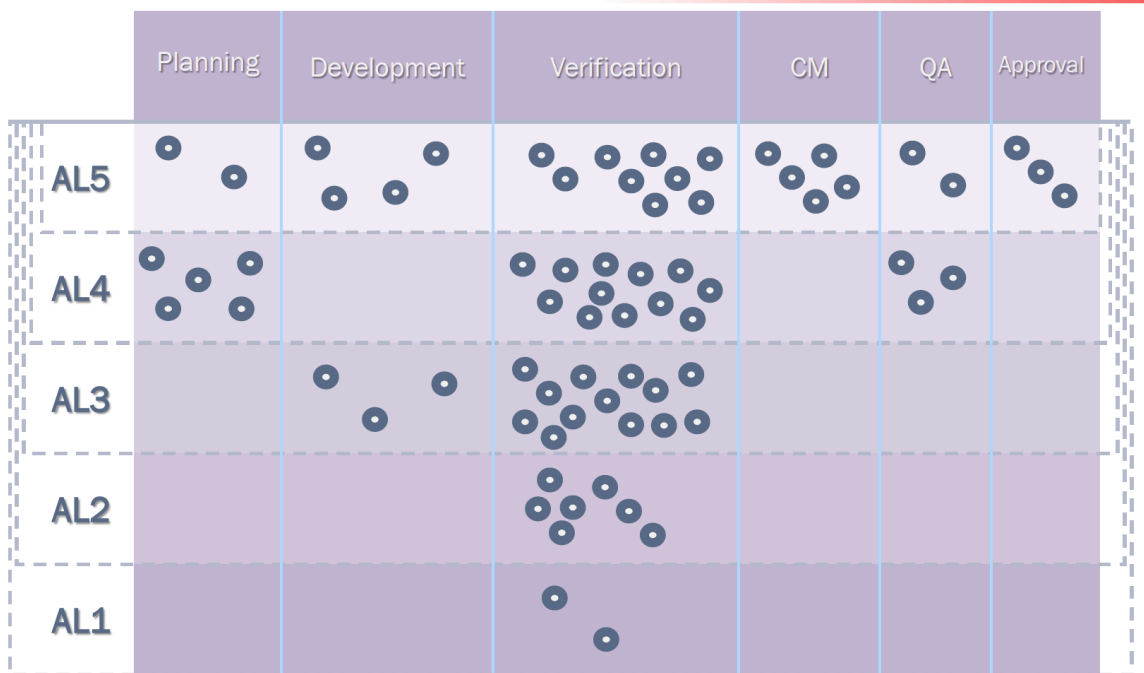
16

Software Assurance (Certification) DO-178C/ED-12C - DO-278/ED-109A examples

- It is a methodological guide based on good practices to develop a critical software
 - It explains **"What to do"** but it doesn't explain **"How to do"**
 - It explains the **objectives** but not the **methods**
- It leads to:
 1. Write what you are doing (Plan)
 2. Do what you have written (Processes)
 3. Keep evidence of what you have done (traceability)
 4. Compare what was done with what is written (Audit)
 5. Correct deviations
 6. Improve the method

17

Software Assurance DO-278/ED-109A Objectives repartition example



18

Software Assurance (Certification) DO-178C/ED-12C example

Category	Potential Failure Condition Effect	Likelihood of Occurrence	Assurance Level	Number of required Process Objectives/Objectives to satisfy with independence
Catastrophic	Failure conditions which would result in multiple fatalities, usually with the loss of the airplane	Extremely improbable	A	71/30
Hazardous/Severe major	Failure condition would significantly reduce the ability of the crew and/or aircraft capabilities to cope with adverse operating conditions: Large reduction of safety margin, Physical distress or excessive crew workload, serious or fatal injury to a relatively small number of the occupants other than the flight crew	Extremely remote	B	69/18
Major	Failure condition would significantly reduce the ability of the crew and/or aircraft capabilities to cope for adverse operating conditions	Remote	C	62/5
Minor	Failure conditions, which would not significantly reduce airplane safety and which involve crew actions that are well within their capabilities	Reasonably probable	D	26/2
No safety effect	Failure conditions that will have no effect on the safety (operational capability not affected)	Probable	E	0/0

19

Software Assurance (Certification) DO-178C/ED-12C Table example

TABLE A-2: SOFTWARE DEVELOPMENT PROCESSES

	Objective		Activity	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref		Ref	A	B	C	D	Data Item	Ref	A	B	C
1	High-level requirements are developed.	5.1.1.a	5.1.2.a 5.1.2.b 5.1.2.c 5.1.2.d 5.1.2.e 5.1.2.f 5.1.2.g 5.1.2.j 5.5.a	O	O	O	O	Software Requirements Data	11.9	①	①	①	①
								Trace Data	11.21	①	①	①	①
2	Derived high-level requirements are defined and provided to the system processes, including the system safety assessment process.	5.1.1.b	5.1.2.h 5.1.2.i	O	O	O	O	Software Requirements Data	11.9	①	①	①	①
3	Software architecture is developed.	5.2.1.a	5.2.2.a 5.2.2.d	O	O	O	O	Design Description	11.10	①	①	①	②
4	Low-level requirements are developed.	5.2.1.a	5.2.2.a 5.2.2.e 5.2.2.f 5.2.2.g 5.2.3.a 5.2.3.b 5.2.4.a 5.2.4.b 5.2.4.c 5.5.b	O	O	O		Design Description	11.10	①	①	①	
								Trace Data	11.21	①	①	①	

5.2

SOFTWARE DESIGN PROCESS

The high-level requirements are refined through one or more iterations in the software design process to develop the software architecture and the low-level requirements that can be used to implement Source Code.

5.2.1

Software Design Process Objectives

The objectives of the software design process are:

- The software architecture and low-level requirements are developed from the high-level requirements.
- Derived low-level requirements are defined and provided to the system processes, including the system safety assessment process.

5.2.2

Software Design Process Activities

The software design process inputs are the Software Requirements Data, the Software Development Plan, and the Software Design Standards. When the planned transition criteria have been satisfied, the high-level requirements are used in the design process to develop software architecture and low-level requirements. This may involve one or more lower levels of requirements.

The primary output of the process is the Design Description (see 11.10) which includes the software architecture and the low-level requirements.

The software design process is complete when its objectives and the objectives of the integral processes associated with it are satisfied. Activities for this process include:

- Low-level requirements and software architecture developed during the software design process should conform to the Software Design Standards and be traceable, verifiable, and consistent.
- Derived low-level requirements and the reason for their existence should be defined and analyzed to ensure that the higher level requirements are not compromised.
- Software design process activities could introduce possible modes of failure into the software or, conversely, preclude others. The use of partitioning or other architectural means in the software design may alter the software level assignment for some components of the software. In such cases, additional data should be defined as derived requirements and provided to the system processes, including the system safety assessment process.
- Interfaces between software components, in the form of data flow and control flow, should be defined to be consistent between the components.
- Control flow and data flow should be monitored when safety-related requirements dictate, for example, watchdog timers, reasonableness-checks, and cross-channel comparisons.

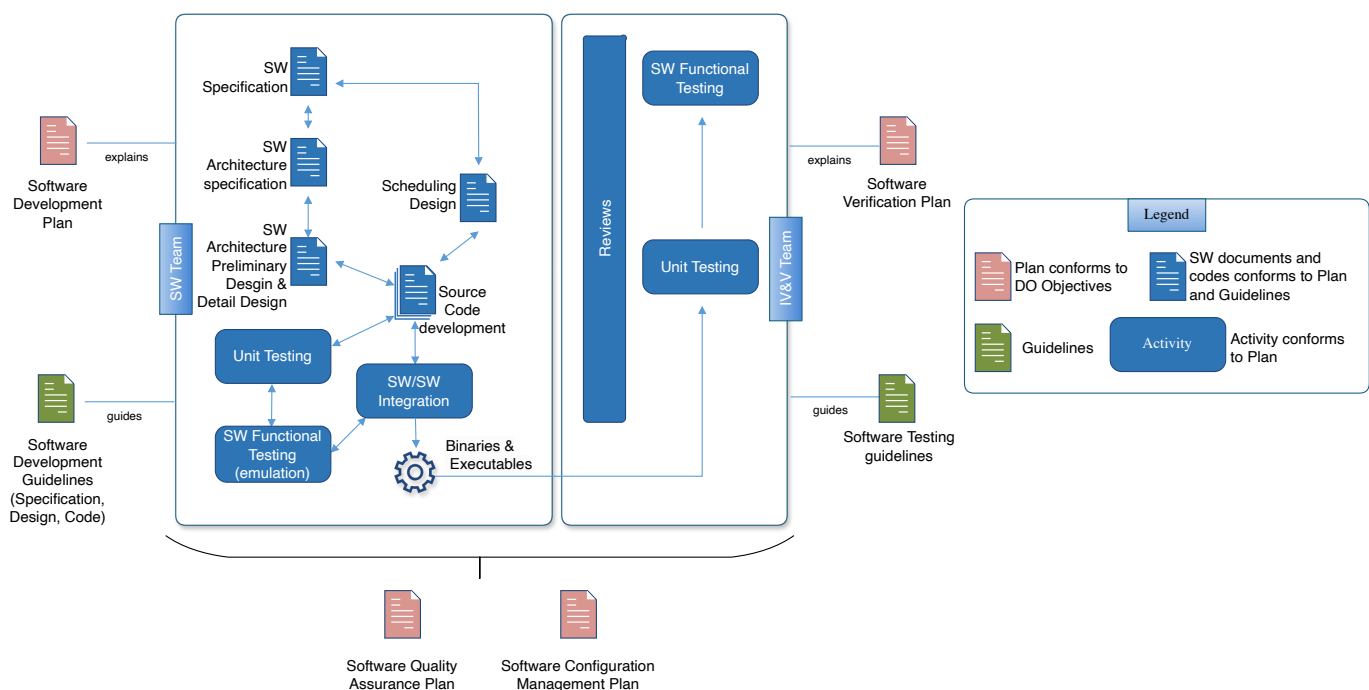
20

Necessary Definitions to discuss Software Confidence: Validation & Verification ?

- **Validation**, from [DO-178C/ED-12C], is the process of ensuring that requirements are correct and complete. **We implement the right system.**
- **Verification**, from [DO-178C/ED-12C], is the process of ensuring that the system has been implemented as specified. **We implement the system right.**

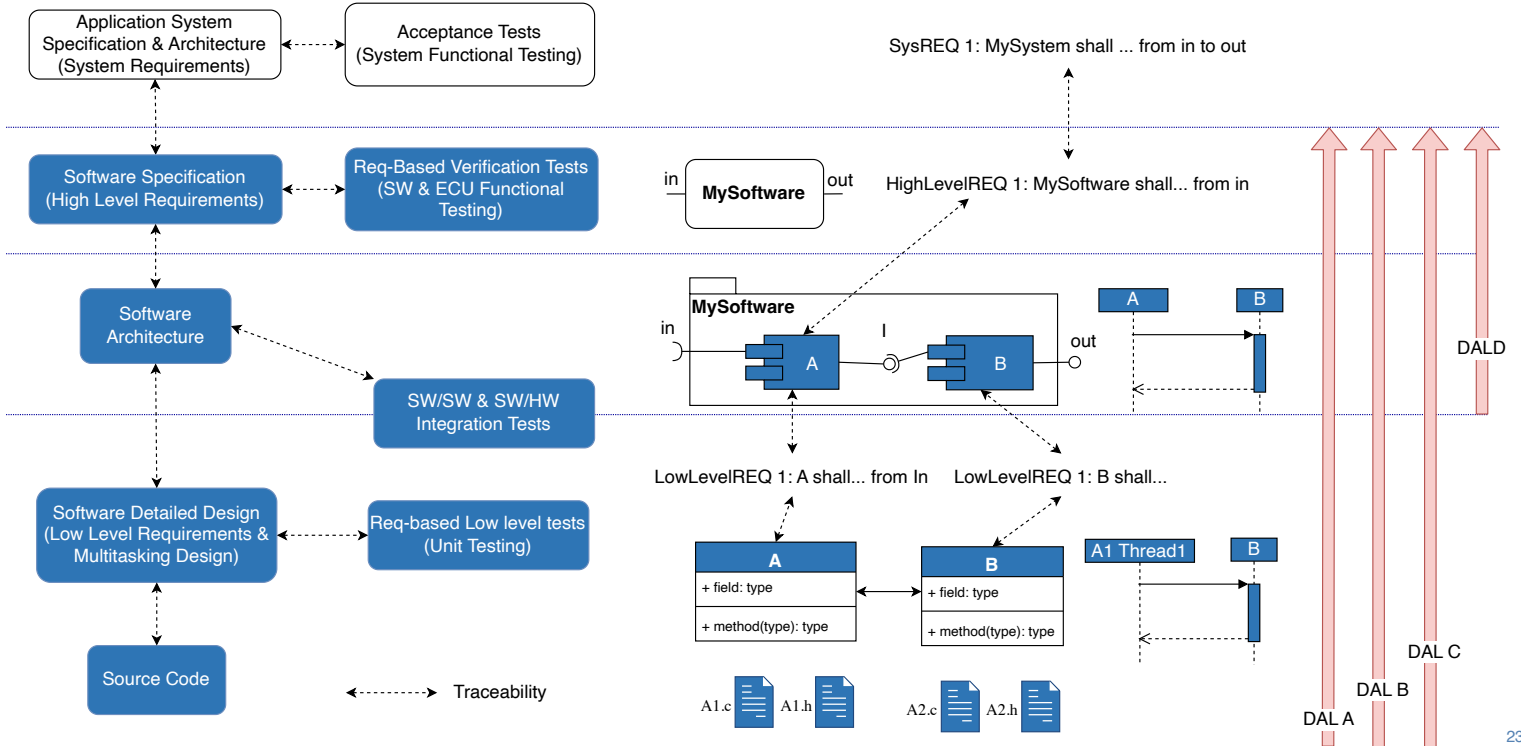
21

Concrete System Engineering for safety software with DO-178C/ED-12C certification objectives (1/3)



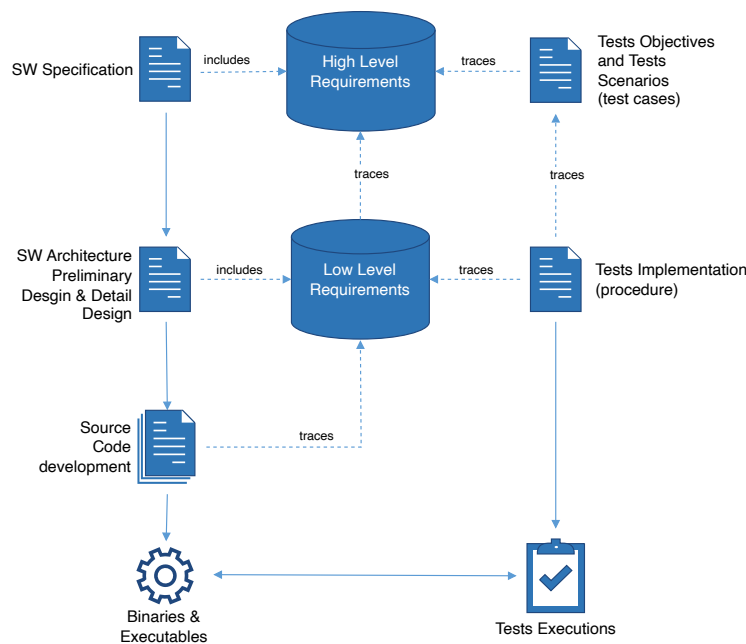
22

Concrete System Engineering for safety software with DO-178C/ED-12C certification objectives (2/3)



23

Concrete System Engineering for safety software with DO-178C/ED-12C certification objectives (3/3)



24



Capture requirements

- Requirements should not provide any solution but are the direct translation of user's needs into formalized and technical terms
- Requirements aim at:
 - Define the **the what** and not the how
 - Facilitating the communication
 - Set a baseline for validation and verification activities

25

Characteristics of Good Requirement

- **Reference Data dictionary and interfaces:** reference the defined input/outputs
- **Atomic:** each requirement shall be a single requirement
- **Complete:** contains all necessary information, stand alone requirement
- **Concise and Consistent:** describes clearly what a software shall do
- **Implementation free:** should state what is required without how to implement
- **Traceable:** uniquely identified to allow references (through abstraction levels)
- **Verifiable:** should be written such that it can be verified by review, analysis or tests. For that it only references observable external interfaces and not internal states/variables

26

Requirement example

27

Design approaches overview

Can you illustrate some design approaches and languages?

Summary

1. Design Approaches Overview



28

Design the solution

- Design activities aims at designing the technical solution to satisfy requirements: **define the how**
- Design aims at:
 - Defining the structural and dynamic architecture
 - Providing the information to allow developers to implement

29

Characteristics of Good Design

- **Abstraction**: use a representation similar to its meaning while hiding the implementation details (execution platform abstraction)
- **Determinism**
- **Modularity**: separate the features/functions in modules/components
- **Cohesion**: sequential cohesion, productions/consumptions, temporal cohesion
- **Loose coupling**: minimize coupling by eliminating unnecessary relationship
 - Content coupling: one component affects the working of another one
 - Common coupling: two components use the same global data
 - External coupling: communication with a common communication means (file, database)
 - Control coupling: one component controls the execution of another one
 - Data Structure coupling

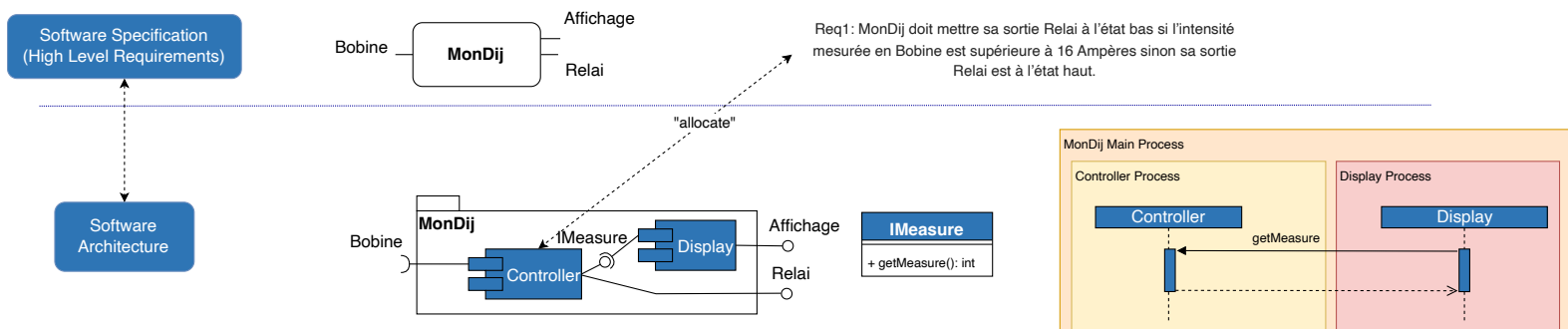
30

Characteristics of Good Design

- **Maintainable**
- **Robust**: interrupt functionality, error and exception handling
- **Document design decisions**: documentation the rational for decisions made during the design process
- **Document safety features**: ex watchdog
- **Document cybersecurity features**: ex authentication
- **Reviewed**
- **Complete**: Nominal, temporal and error behavior descriptions
- **Testable**: Error of fault logging, Observability, Tests points, Access to interface
- **Enabler**: Allow the implementation without doubts, implicit choices

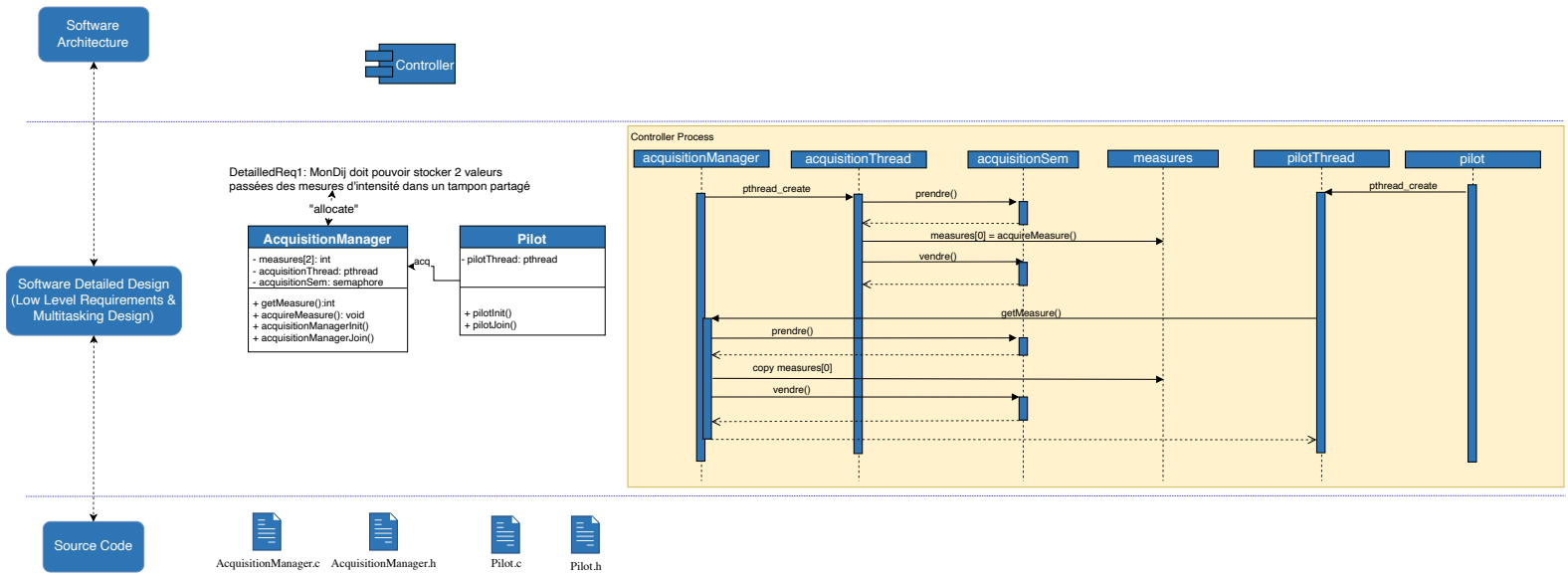
31

Disjoncteur électronique



32

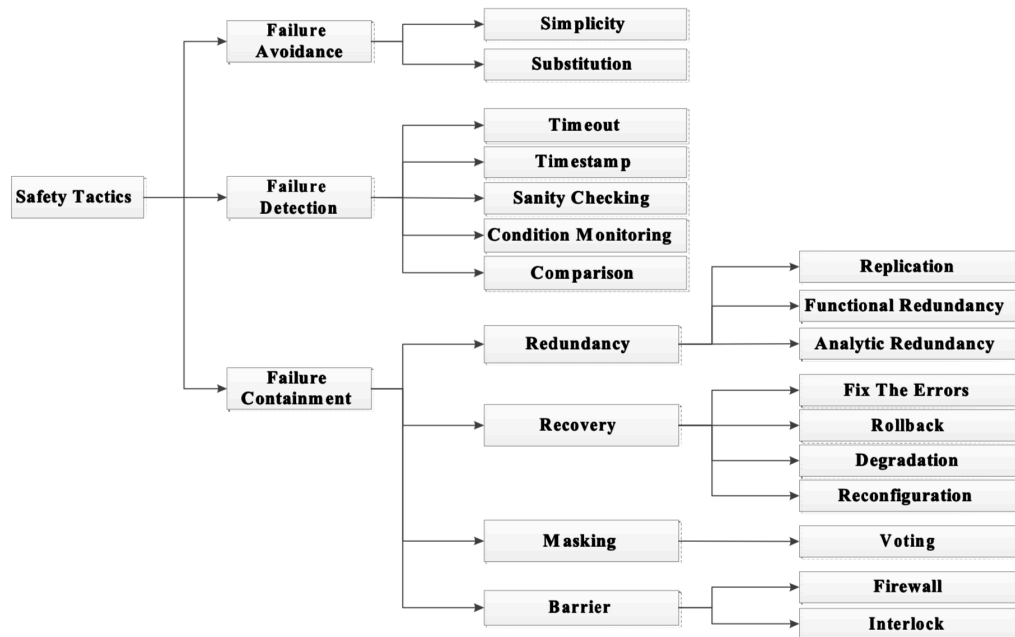
Disjoncteur électronique



<https://www.plantuml.com/> est un outil pratique pour rapidement faire des diagrammes UML

33

Safety Design Tactics



http://www.christian-rehn.de/wp-content/uploads/downloads/2010/06/seminar_safe_sec.pdf

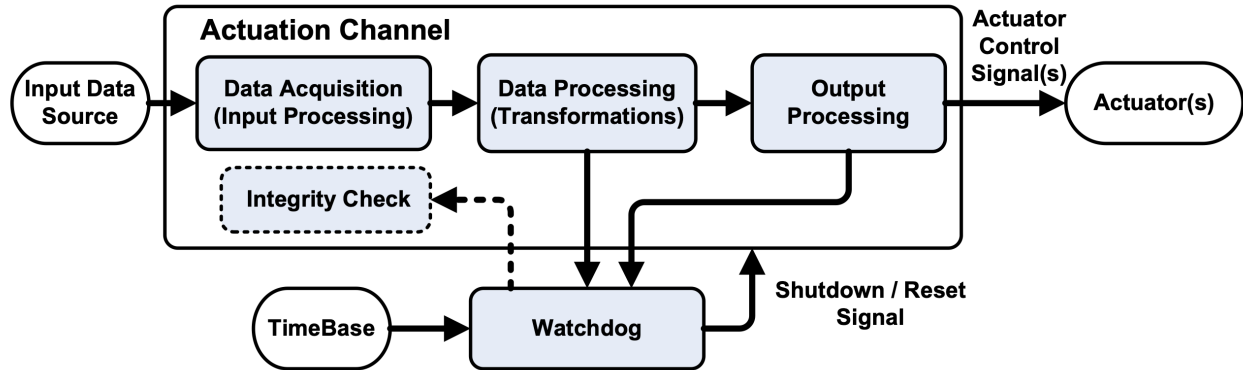
<https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.62.2420&rep=rep1&type=pdf>

Safety Design Tactics

Tactic	Description
Simplicity	Keep it simple to minimize errors!
Substitution	Develop in parallel with two independent teams
Timeout	Detect that an order, a data or a message is no more valid: a duration after which an error is triggered
Timestamp	Associate to an event a date to be able to detect a timeout, an incoherency
Sanity Checking	Test the integrity of an order, a data or a message (e.g. Cyclic redundancy check)
Condition Monitoring	Monitor a set of criteria (e.g. CPU, RAM, requests) to detect an abnormal behavior (black box monitoring)
Comparison	Compare the output from a valid range
Replication	Duplicate the software
Functional Redundancy	Duplicate the functions (and the hardware)
Analytic Redundancy	Diversified the computation with a mathematical models executing in parallel of sensors and actuators
Fix the errors	Correct online a data (bit replacement)
Rollback	Cancel an action to retrieve a previous state
Degradation / Reconfiguration	Stop/Reconfigure a function
Voting	2 out of 3
Firewall	Filter requests
Interlock	Ensure sequential accesses

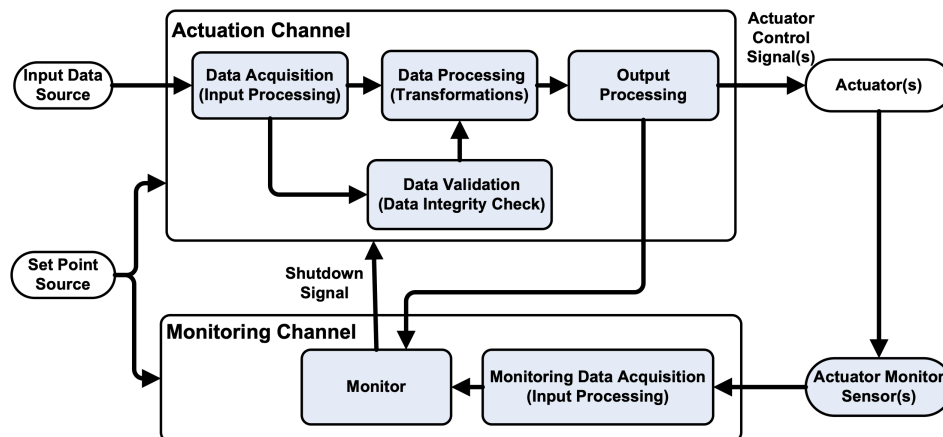
Safety Design Tactics: Watchdog

- **Safety Design Tactics:** Failure Detection - Timeout
- **Context:** Detect if a software is still alive, or too long execution of a function/software (deadlock detection)
- **Description:**
 - A timer is periodically armed to monitor liveness of sw/function.
- **Safety improvement:**
 - Reliability: only for undetected loss



36

Safety Design Tactics: Command and Monitoring



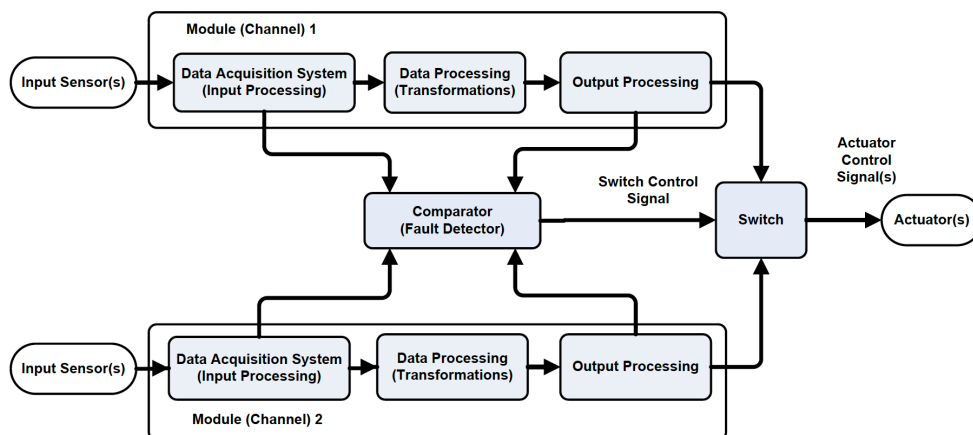
37

Safety Design Tactics: Command and Monitoring

- **Safety Design Tactics:** Failure detection - Sanity Check/Monitoring
- **Context:** Detect of loss, erroneous behavior. Safety monitoring may be on the same HW, in this case, it does not protect against hardware failure.
- **Description:**
 - One channel is responsible for performing the end-to-end actuation, while the monitor channel compares the output of the actuator channel against the expected value and shuts the actuator off, if a discrepancy is found.
 - This pattern assumes that loss of the actuation function is a less critical condition.
 - The monitoring channel usually contains a reduced version of the actuation algorithm and is driven by a separate set of sensors to avoid common mode failure.
 - The monitoring channel differs from the actuator channel such that if it contains any fault, the actuation channel continues to operate properly.
- **Safety improvement:**
 - Reliability: for random and systematic fault of the command channel

38

Safety Design Tactics: Homogenous Duplex Pattern



Design Patterns for Safety-Critical Embedded Systems

39

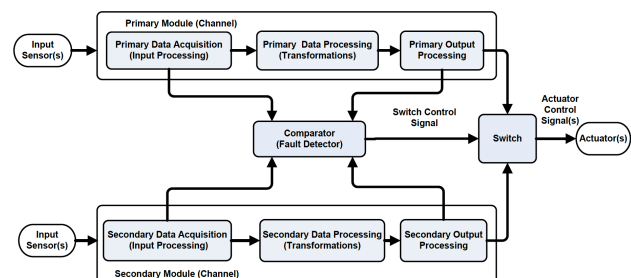
Safety Design Tactics: Homogenous Duplex Pattern

- **Safety Tactics:** Failure detection & Failure Containment - Sanity Check/Monitoring/Redundancy/Masking
- **Context:**
 - Detection of computing corruption, input corruption and input loss for High random failure rate
- **Other Names:** Master/Slave
- **Description:**
 - The comparator identifies the faulty module (channel) in the presence of a fault in one of the two channels, and generates the instruction that controls the switch.
 - The faulty module identification is: either by Acceptance Test (performs a check on the two channels by checking for input valid data within a given range and by checking the output signals from the two modules whether they are valid or not.) or by hardware testing (auto-tests)
- **Safety improvement:**
 - Reliability: for random failure only
 - Availability: by redundancy but the comparator is a single point of failure
 - Maintainability: one channel may be updated whereas others execute
- **Comments:** can be purely software with different version of the same software

40

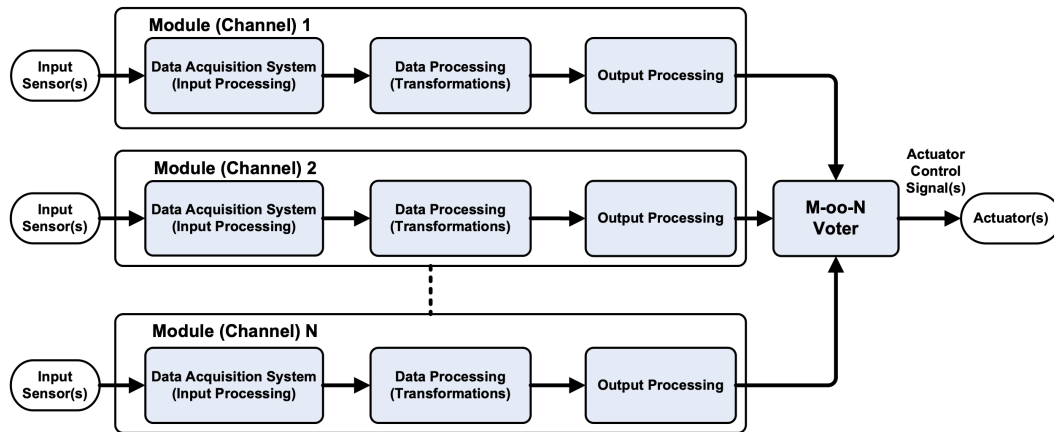
Safety Design Tactics: Heterogeneous Duplex Pattern

- **Safety Tactics:** Failure detection & Failure Containment - Sanity Check/Monitoring/Redundancy/Masking
- **Context:**
 - Detection of computing corruption, input corruption and input loss: High random failure rate and High systematic failure rate
- **Other Names:** Master/Slave
- **Description:**
 - Same as previous pattern
 - Software and Hardware diversified (dissimilarity)
- **Safety improvement:**
 - Reliability: for random failure only and systematic fault handled
 - Availability: by redundancy but the comparator is a single point of failure
 - Maintainability: one channel may be updated whereas others execute



41

Safety Design Tactics: M out of N (MooN)



Design Patterns for Safety-Critical Embedded Systems

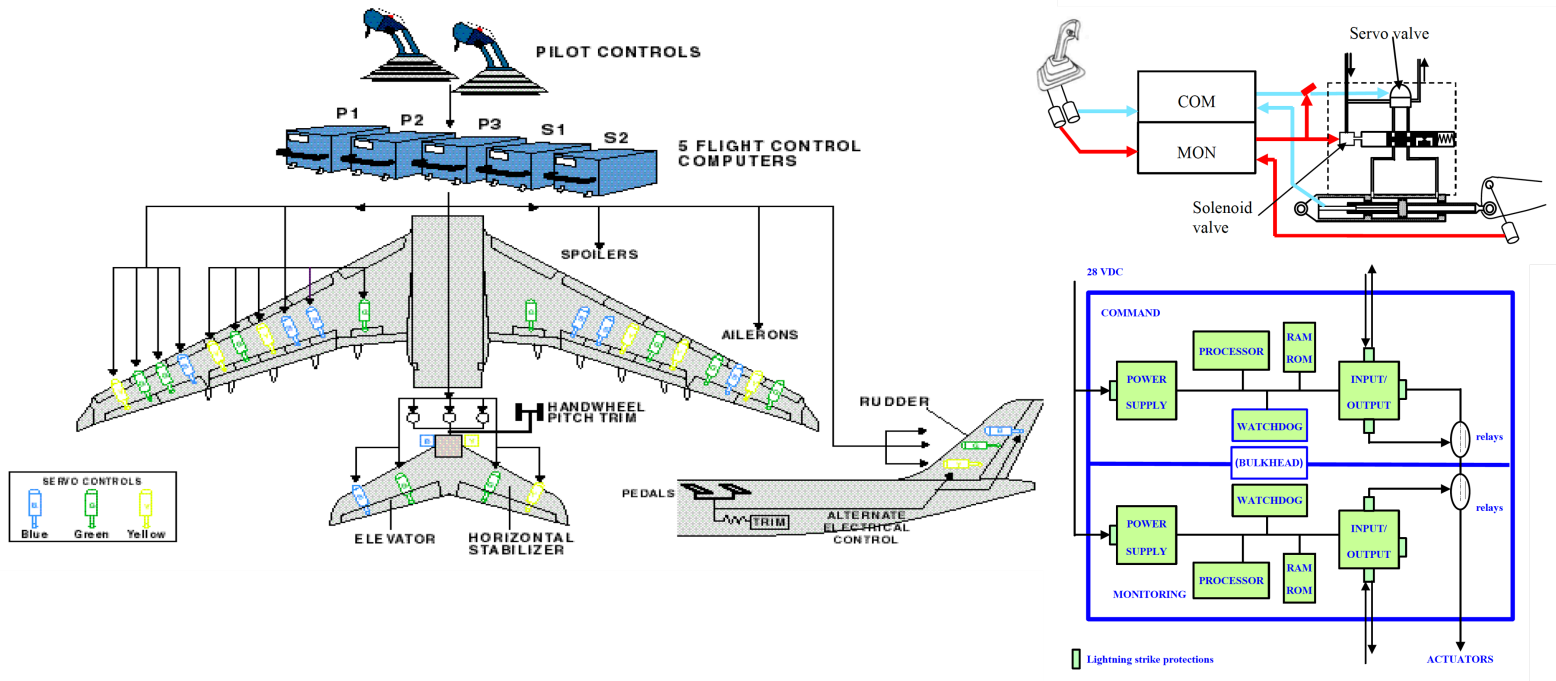
42

Safety Design Tactics: M out of N (MooN)

- **Safety Tactics:** Failure detection & Failure Containment - Sanity Check/Monitoring/Redundancy/Voting
- **Context:**
 - Extension of Homogenous/Heterogenous pattern: N similar/dissimilar instances running on similar/dissimilar HW
- **Other Names:** 2oo2, 2oo3, etc.
- **Description:**
 - the output will be the result a voting scheme of the outputs of N redundant channels. If one instance has different output from the other one, then instance is killed and reboot.
 - Software and Hardware diversified (dissimilarity)
- **Safety improvement:**
 - Detection of corruption and loss.
 - Reliability: for random failure only and systematic fault handled with heterogeneity
 - Availability: by redundancy but the comparator is a single point of failure
 - Maintainability: one channel may be updated whereas others execute

43

Airbus A340-600 System Architecture Example (1/2)

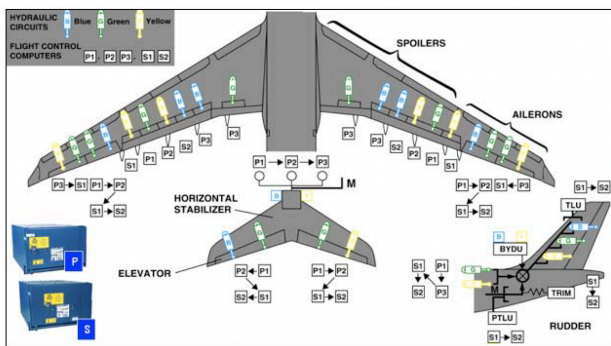


A Process Toward Total Dependability – Airbus Fly-by-Wire Paradigm

Architectures innovantes de systèmes de commandes de vol

44

Airbus A340-600 System Architecture Example (2/2)



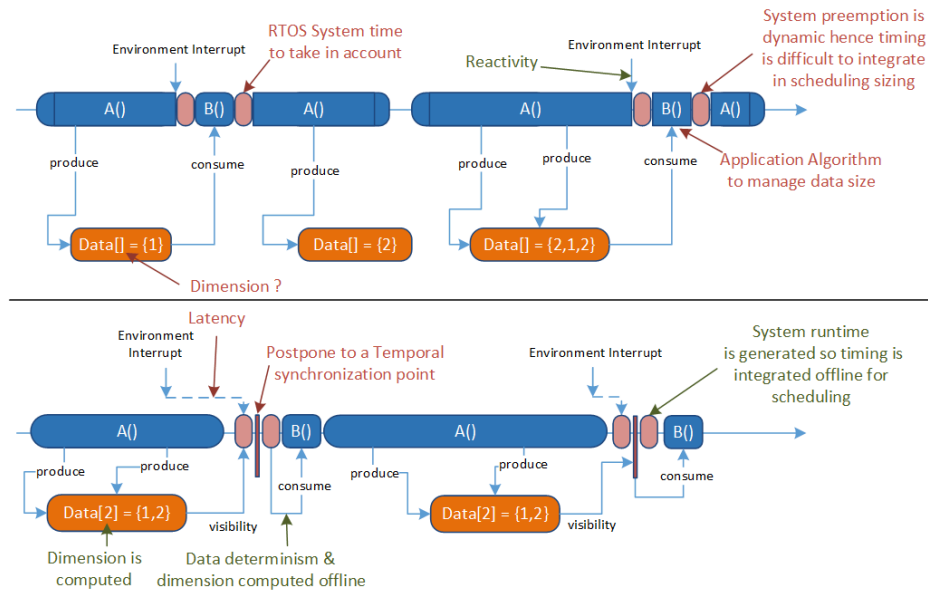
The computers and actuators are redundant. This is illustrated by the A340-600 pitch control (Fig. 2, left and right elevator, plus Trimable Horizontal Stabilizer - THS). Four command and monitoring computers are used, one is sufficient to control the aircraft. In normal operation, one of the computers (PRIM1) controls the pitch, with one servocontrol pressurized by the Green hydraulic for the left elevator, one pressurized by the Green hydraulic on the right elevator, and by electric motor No. 1 for the THS. The other computers control the other control surfaces. If PRIM1 or one of the actuators that it controls fails, PRIM2 takes over (with the servocontrols pressurized by the Blue hydraulic on left elevator, yellow on right side, and with THS motor No. 2). Following same failure method, PRIM2 can hand over control to SEC1. Likewise, pitch control can be passed from one SEC to the other depending on the number of control surfaces that one of these computers can handle.

Note that 3 computers would be sufficient to meet the safety objectives. The additional computer is fully justified by operational constraints: it is desirable to be able to tolerate a take-off with one computer failed. This defines the Minimum Equipment List (MEL).

A Process Toward Total Dependability – Airbus Fly-by-Wire Paradigm

45

Design approaches for determinism (1/3)



46

Design approaches for determinism (2/3)

- **Event-Trigger (asynchronous approach):** determinism is not provided by construction
 - - Processing are not bounded: unpredictable end to end execution time which depends of the hardware performances
 - - Data communication buffer are not bounded offline: unpredictable overflow
 - - Temporal constraints are traducted to function priorities: determinism is not ensure during integration
 - - Developers/Designers shall demonstrate timing partitioning
 - - CPU workload
 - + Reactivity

47

Design approaches for determinism (3/3)

- **Time-Trigger (synchronous approach):**
 - + Temporal constraints shall be given at the design step
 - + Communicated data shall be time stamped at design
 - + All processing are bounded between two instants
 - + All data transfers are done at a temporal synchronization point (Observability): timing partitioning by construction
 - + CPU Load leveled
 - - Latency

48

Design Approaches: Event trigger approach example (1/2)

- SDL, SADT, SART, ...
- **UML** has been designed to specify and analyze the architecture of large software
 - Class diagram and component diagram to define the architecture
 - Sequence diagram to describe the event sequences
 - Activity and State Diagram to define the behaviors
 - A set of editor and code generator
 - Based on asynchronous event trigger computation model
 - No formal execution
 - No formal time model
 - No Safety model features

49

UML

Design concepts	Approach Evaluation	Comments
Determinism	-	Execution model is not formally defined.
Separation of concerns	+	Declarative approach. Concepts to describe the designs but no formal execution semantics
Execution support abstraction	-	Execution model is not formally defined. No execution environment
Completeness of the description	++	Multiple level declaration (structural design, behavioral design, multitasking, hardware allocation) but no timing extension
Temporal description	-	No specific language artifacts to manage temporal description
Approval/Certification/Qualification	--	

- You can use UML as a **graphical standard** in the Software Architecture Description with specific guidelines
 - Components diagrams to describe the architecture and the allocations
 - Sequences diagrams to describe the thread, the sequence of actions

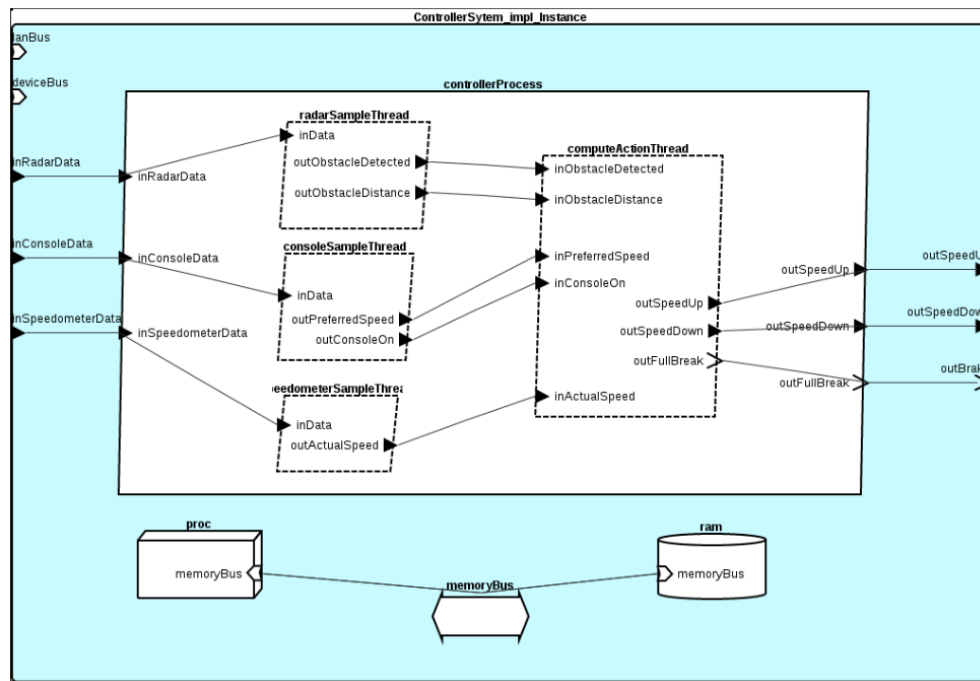
50

Design Approaches: Event trigger and Time trigger approach example (2/2)

- AADL** has been designed to specify and analyze the architecture of large embedded real-time systems
 - Component based model: Architecture Description Language (definition of the thread interfaces)
 - A set of **editor, analyzer and code generator (e.g. Ocarina)**
 - Based on both synchronous and asynchronous event trigger computation model
 - AADL Standard specifies an execution model as a virtual runtime environment

51

AADL Cruise Control Example



Extracted from "Formal semantics of behavior specifications in the architecture analysis and design language standard"

52

AADL Cruise Control Example: timing properties definition

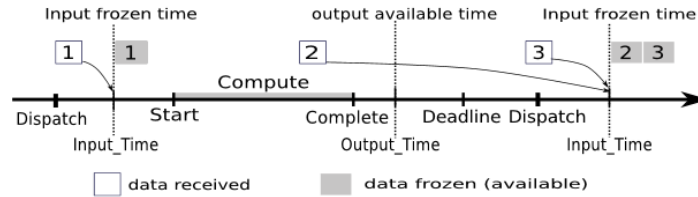
```
[2] thread [2] implementation ComputeActionThread.impl
  [2] properties
    -- periodic thread
    Dispatch_Protocol => Periodic;
    Period => 50 ms;
    -- thread deadline
    Deadline => 40 ms;
    -- thread WCET
    Compute_Execution_Time => 20 ms;
  [2] end ComputeActionThread;
```

Extracted from "Formal semantics of behavior specifications in the architecture analysis and design language standard"

53

AADL

- AADL timing execution model: A thread is activated to perform a computation at start time, and has to be finished before the deadline. A complete event is sent at the end of the execution. The received inputs are frozen at a specified time (Input Time), by default the dispatch time.



Extracted from "Formal semantics of behavior specifications in the architecture analysis and design language standard"

54

AADL

- Example of AADL Thread
 - Periodic thread: thread is awake periodically.
 - Sporadic thread: thread is awake when receiving events/messages. Message timing properties are defined (lower and upper bound).
 - Aperiodic thread: thread is awake when receiving messages. No timing properties defined for messages

55

AADL

Design concepts	Approach Evaluation	Comments
Determinism	+	Execution model is formally defined by a specific annex.
Separation of concerns	+	Declarative approach. Concepts to describe the multitasking design: structural view, no dynamic view.
Execution support abstraction	+	Execution model is based on a virtual execution environment implemented in specific middlewares
Completeness of the description	+/-	Multiple level declaration (component, partitions, threads, hardware allocation) but thread behaviors are described by C codes.
Temporal description	+	Declaration of timing properties (periodic behaviors). Time is defined from common clocks (s, ms, us).
Approval/Certification/Qualification	--	No certified middlewares

- You can use AADL as a **multitasking design language** in the Software Architecture Description to describe a structural view of the partitions, threads and to generate operating system multitasking configurations (e.g. API calls)

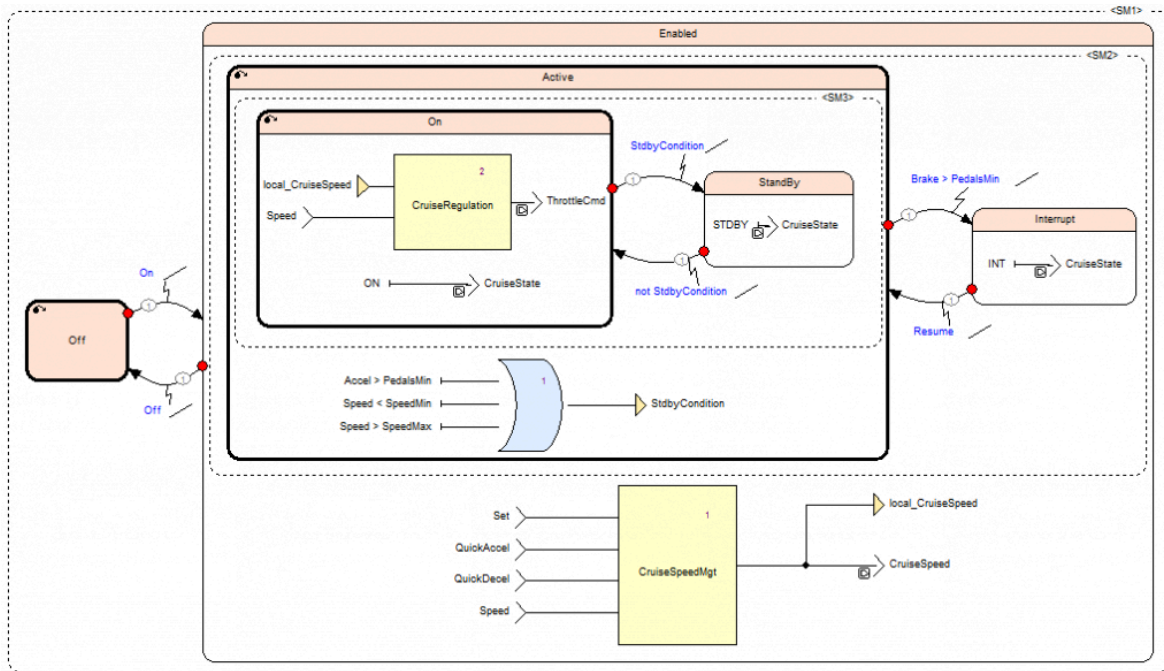
56

Design Approaches: Time trigger approaches

- Lustre & Esterel based model**
 - Synchronous approach: execution is immediate and activities are periodic
 - Formal model of time, Formal simulation
 - Industrial success: SCADE (Esterel-Technologies company) is used for programming critical control software (e.g., planes, nuclear plants).
 - No Partitioning model
 - No specific embedded kernel
 - No specific kernel and no scheduling generator among multiple synchronous application (Pb with mixing applicaiton on the same CPU)

57

Scade Suite Cruise Control Example



Extracted from "SCADE: Industrial Success of a Synchronous Language and its Future Challenges (2007) by Amar Bouali"

58

Lustre & Esterel based model

Design concepts	Approach Evaluation	Comments
Determinism	++	Synchronous computation model is determinist
Separation of concerns	++	Declarative approach, dataflow and stateflow descriptions, automatic verification, automatic qualified code generation
Execution support abstraction	+	As execution is immediate, hardware independence
Completeness of the description	+	Description of all functional behaviors (dataflow and state flow descriptions)
Temporal description	+	Clocks definitions and synchronous state machine description
Approval/Certification/Qualification	++	Code generator from Scade models that has been qualified as a development tool for DO-178B software up to Level A, DO-178C/DO-330 at TQL-1, certified for IEC 61508 at SIL 3, and for EN 50128 at SIL 3/4, and qualified for ISO 26262 software up to ASIL D

- You can use Lustre & Esterel based model for a model based approach and automatic certified functional generation of code without Operating System thread specific usage (no generated API calls)

59

Design Approaches: Time trigger approaches

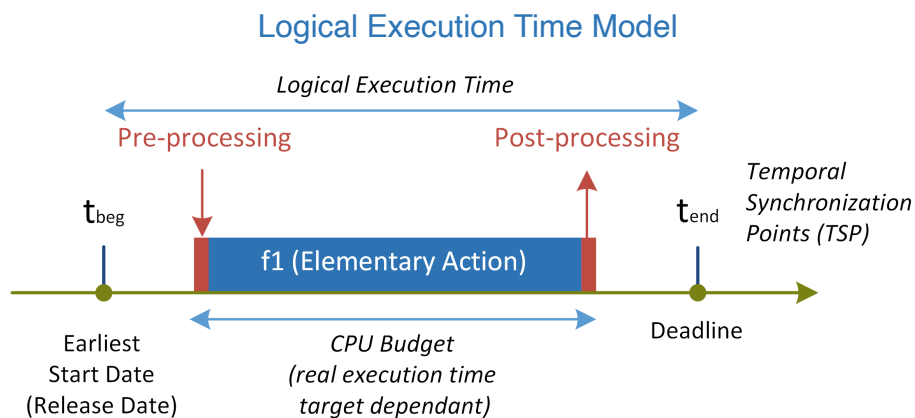
- Giotto

- Code interpretation on embedded
- Safety oriented

- Psy Approach

- Synchronous & Asynchronous approach: execution is not immediate and activities are not only periodic
- Formal model of time, Formal simulation, Safety oriented
- Industrial implementation
- Specific Kernel for embedded systems
- Code and safety scheduling plan generation

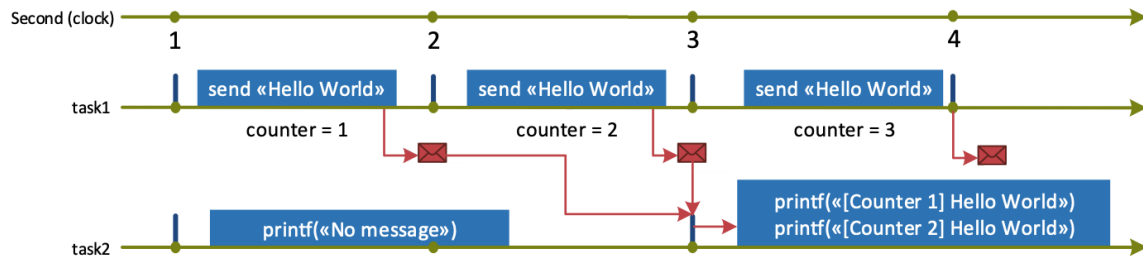
60



61

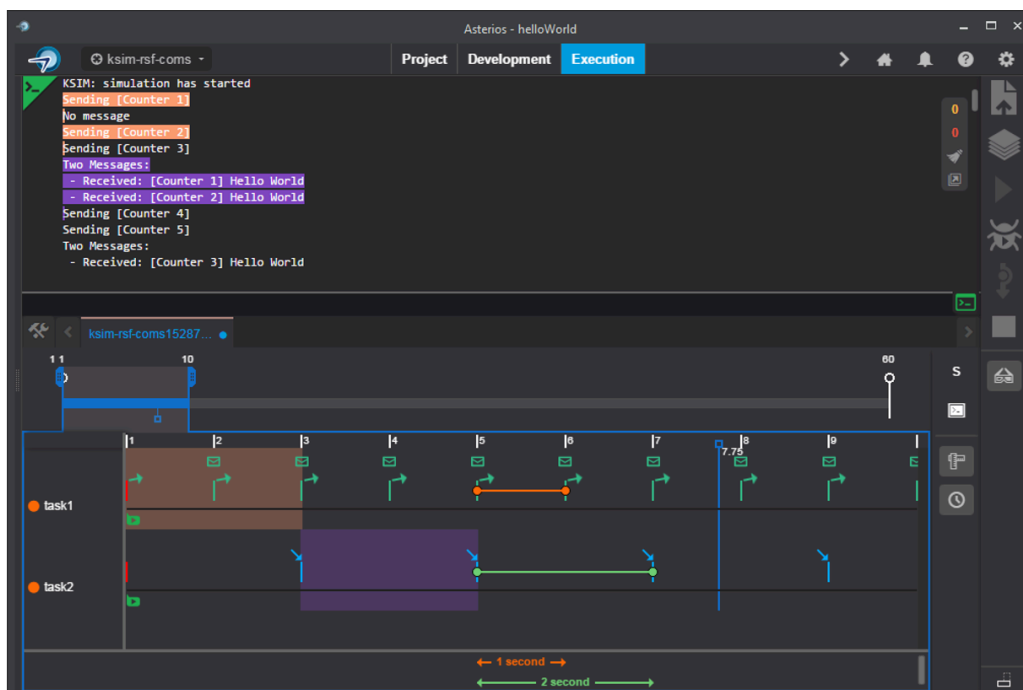
Logical Execution Time Model - Psy Example

- Hello World specification:
 - Produce « Hello World \n» string with a counter value incremented each second and in one second to a consumer
 - Print all the produced strings each two seconds and in two seconds
 - If no string produced, print « No message »
- Hello World design:
 - All the early start dates and deadlines are instants of the clock « second » which triggers each second,
 - Two tasks: one for the producer, another for the consumer
 - The communicated data are visible at the instants of the clock « second »



62

Psy Execution with Asterios Developer



63

Psy Approach

Design concepts	Approach Evaluation	Comments
Determinism	++	Psy computation model is determinist
Separation of concerns	+	Declarative approach and automatic verification. Behviors are described in C.
Execution support abstraction	++	Psy computation model is not dependant of the scheduling and hardware cores available. A Psy specific kernel is provided.
Completeness of the description	+	Description of all functional and temporal behaviors
Temporal description	++	Clocks and constraint definition
CPU load management	++	Scheduling is computed at design level (offline) from the clocks and the timing constraints
Approval/Certification/Qualification	+/-	DAL A certification in progress for aeronautic. Acquired for nuclear industry.

- You can use Psy based model for a model based approach and automatic Psy Kernel specific API calls

64

Safety-related Multitasking Implementation Concerns

Can you illustrate implementation concepts and APIs?

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Langage generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



65

Summary



1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Language generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points

66

Safety-related Multitasking Implementation Concerns

- **In a safety-critical system confidence is linked to determinism** which implies mastering during the implementation phase:
 - **The correctness of the program's computations**: Static Analysis (Clang Static analyzer, Frama-C)
 - **The race condition**: the execution path is known and the same from one execution to another through thread synchronization
 - **The temporal partitioning**: the instant at which the computing resources become available and the duration of availability (Jitter management)
 - Time-bound system: the time taken to perform the computation is important. The influence of CPU sharing, memory sharing and external interrupts must be fully predictable.
 - Deterministic scheduling with single and multicores
 - Deterministic interruption management (minimize the preemption duration)
 - Worst case execution time mastering: cache management, page management
 - **The spatial partitioning**: fault confinement, integrity of code, data and resources accesses
- The determinism depends a lot of the operating system design and implementation

67



Why Partitioning?

- **Needs: co-hosted several software components on the same target hardware (target computer)**
- **Partitioning** can be described as : "a technique for providing isolation between software components to contain and/or isolate faults and potentially reduce the effort of the software verification process" [DO-178C/ED-12C]
- An architectural technique to provide necessary separation and independence of functions
- Failures of hardware unique to a partitioned software component should not cause adverse effects on other partitioned software components
- A partitioned software component should not be allowed to contaminate another partitioned software component's code, input/output or data storage areas: **Spatial Partitioning**
- A partitioned software component should be allowed to consume shared processor resources only during its scheduled period of execution: **Timing Partitioning**
 - The goal of temporal partitioning is to ensure that functions in one partition do not disturb the timing of events in other partitions (one partition monopolizes the CPU, crashes the system, add jitter)

68

Why Partitioning?

- For example, if two functions without any functional links (communications, interactions) share a common ECU (Electronic Control Unit) based on a priority based:
 - You have to analyze the behavior of each function and test them individually
 - Since the execution of one function can delay the execution of the other functions, argument concerning the correct temporal behavior must be based on an analysis of both functions: **complex analyses**
 - Each functions may operate correctly when executing individually on the ECU, but exhibit failures after integration when executing collectively on the ECU, hence nominal and robustness integration tests are needed: **a lot of test cases**

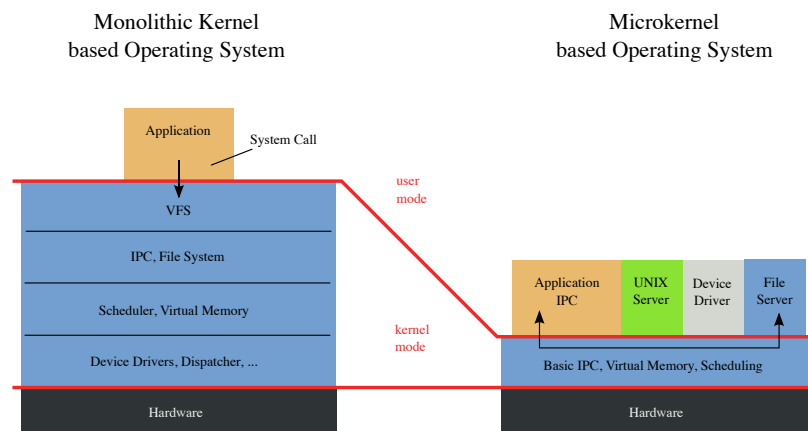
69

Operating system generalities

- The Operating System (OS) kernel **takes a physical resource (such as the processor, or memory, or a disk) and transforms it into a more general, powerful, and easy-to-use virtual form of itself. It virtualizes the cpu, the interruptions, the memory. It allows concurrency. It is a ressource manager that controls the hardware and coordinates its use among the various user codes**
- OS kernel executes in **supervisor mode/kernel space**. Kernel space is a power operating mode that allows the system to interact directly with the hardware
- Several kernel approach exists:
 - **Monolithic kernels**: such as BSD/Unix/Linux/Windows ones that integrates CPU management, memory management, file systems, drivers.

70

Operating system generalities: kernels



<https://en.wikipedia.org/wiki/Microkernel>

71

Operating system generalities: kernels

- Several kernel approach exists:
 - **Microkernels:** the near-minimum amount of software that can provide the mechanisms needed to implement an operating system (OS).
 - Mechanisms include low-level address space management, thread management, and inter-process communication (IPC)
 - Device drivers, protocol stacks and file systems, are typically removed from the microkernel itself and are instead run in user space.
 - <http://www.minix3.org> (POSIX support)
 - <https://sel4.systems>
 - http://www.cs.fsu.edu/~awang/courses/cop5611_s2004/microkernel.pdf

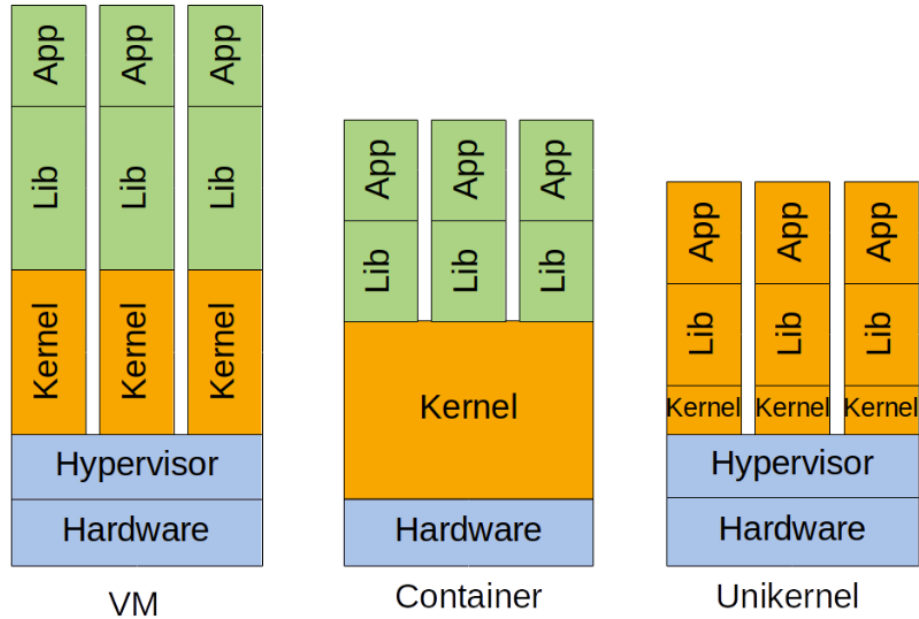
72

Operating system generalities: kernels

- Several kernel approach exists:
 - **Unikernels:** user code is directly integrated with the kernel it is running on.
 - Compiling user source code, along with only the required system calls and drivers, into one executable program using a single address space.
 - Unikernels can only run a single process, thus forking does not exist.
 - The build process results in a complete (virtual) machine image of minimal size that only contains and executes what it absolutely needs to but the resulting virtual machine is executing in supervisor mode and need an hypervisor to share the hardware.
 - <http://osv.io>

73

Operating system generalities: kernels



https://www.researchgate.net/figure/Comparison-of-virtual-machine-container-and-unikernel-system-architecture_fig1_329563819

74

Some good references

- Operating Systems Design and Implementation by Andrew Tannenbaum (<https://www.cs.vu.nl/~ast/>)
- Operating Systems: Three Easy Pieces by Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau (University of Wisconsin-Madison) (<http://pages.cs.wisc.edu/~remzi/OSTEP/>)
- For concrete code example and real time point of view: Solutions temps réel sous Linux: Cas pratique - le Raspberry Pi 3 by Christophe Blaess (<https://www.blaess.fr/christophe/livres/solutions-temps-reel-sous-linux/>)
- Bootlin: Understanding Linux real-time with PREEMPT_RT training (<https://bootlin.com/training/preempt-rt/>)
- The Linux programming interface: A Linux and UNIX System Programming Handbook (<https://man7.org/tlpi/>)

75

System Calls

- **Monolithic kernel and microkernel are used in safety-critical systems**
- Monolithic kernel and microkernel execute in supervisor mode (in **kernel space**)
- **User space** is where most applications run because they don't need supervisor mode power for every operation.
- When a user space code needs additional power, it interacts with the hardware through a **system call** that is conducted by the kernel
- **System calls forms an interface between user programs and the operating system** (allow switching from non-supervisor mode to supervisor mode): Safety
- System calls are function calls from user code into the kernel
- **System calls are abstracted by C library** (in many cases): when you call open, read, write you're calling these wrappers
- **POSIX standard defines a minimal system call interface.**

76

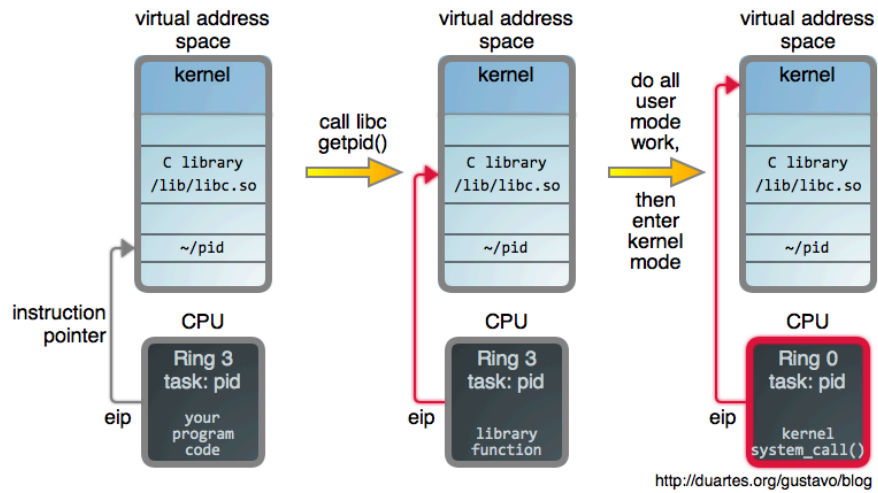
System Calls

POSIX-based kernel Syscall example.

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 int main()
6 {
7     pid_t p = getpid();
8     printf("%d\n", p);
9 }
```

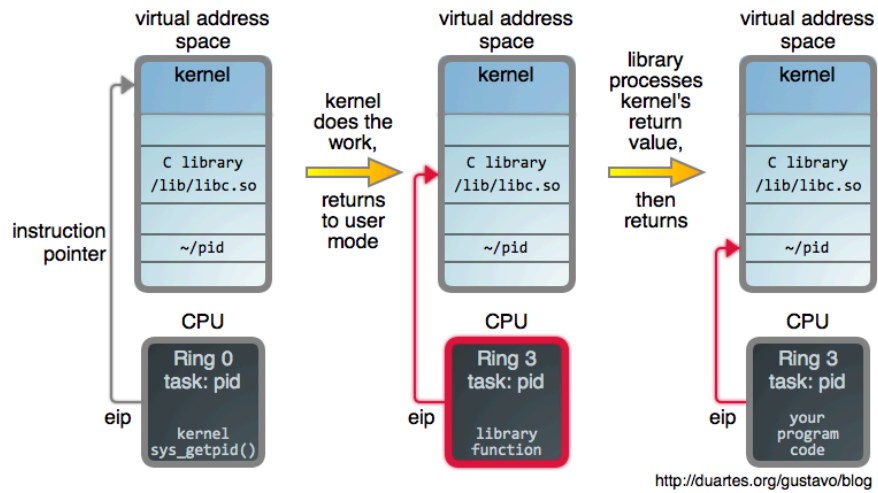
77

System Calls



<https://manybutfinite.com/post/system-calls/>

System Calls



<https://manybutfinite.com/post/system-calls/>

System Calls

```
1 ~/code/x86-os$ strace ./pid
2
3 execve("./pid", ["/.pid"], [/* 20 vars */]) = 0
4 brk(0) = 0x9aa0000
5 access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT (No such file or directory)
6 mmap2(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb7767000
7 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
8 open("/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
9 fstat64(3, {st_mode=S_IFREG|0644, st_size=18056, ...}) = 0
10 mmap2(NULL, 18056, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb7762000
11 close(3) = 0
12
13 [...snip...]
14
15 getpid() = 14678
16 fstat64(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(136, 1), ...}) = 0
17 mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb7766000
18 write(1, "14678\n", 614678
19 ) = 6
20 exit_group(6) = ?
```

<https://manybutfinite.com/post/system-calls/>

80

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Language generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



81

POSIX

- **Portable Operating System Interface (POSIX)**: a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems. POSIX 1003.4 is dedicated for real-time application portability.
- POSIX is a set of standard API not an Operating System
 - **1003.1 (POSIX.1)** System Application Program Interface (kernel)
 - **1003.2** Shell and Utilities
 - **1003.4 (POSIX.4)** Real-time Extensions
 - **1003.7** System Administration
- Detailed description of POSIX API in:
 - [The POSIX Specification: https://pubs.opengroup.org/onlinepubs/9699919799/](https://pubs.opengroup.org/onlinepubs/9699919799/)
 - https://github.com/illinois-cs241/coursebook/blob/pdf_deploy/main.pdf
 - https://perso.telecom-paristech.fr/borde/cours_rt_posix.pdf

82

POSIX 1003.4

FUNCTION	DESCRIPTION
1. Timers	The ability to set and read high resolution internal times
2. Priority Scheduling	A priority-based preemptive scheduling
3. Shared memory	The ability to map common physical space into independent process-specific virtual spaces
4. Real-time files	The ability to create and access files with deterministic performance
5. Semaphores	Efficient synchronization primitives (P and V operations)
6. Interprocess communication	Synchronous and asynchronous message-passing capabilities with facilities for flow and resource control.
7. Asynchronous event notification	A mechanism which provides queuing capabilities, deterministic delivery, and minimal data passing.
8. Process memory locking	The ability to guarantee memory-residence for sections of a process' virtual address space.
9. Asynchronous I/O	The ability to overlap applications processing and I/O operations initiated by the application.
10. I/O Synchronized	The ability to establish assurance of I/O completion at different logical levels of completion.

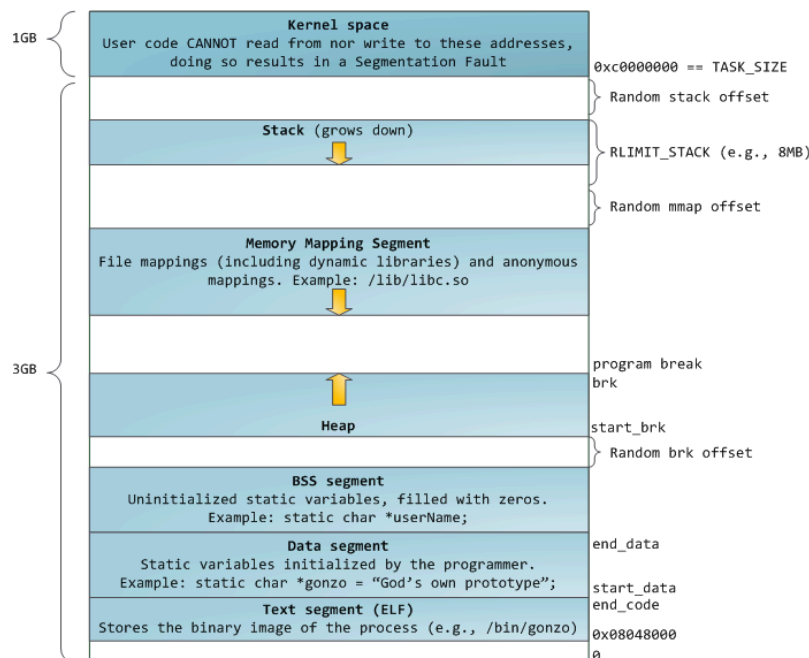
83

POSIX - Process

- POSIX provides **Process** as an isolated space: a partition
- When a process starts, it gets its own address space. Each process gets the following:
 - **A Stack**: the stack is the place where automatically allocated variables and function call return addresses are stored
 - **A Heap**: the heap is a contiguous, expanding region of memory
 - **A Data Segment**: it contains two parts, an initialized data segment (DATA), and an uninitialized segment (BSS)
 - **A Text Segment**: this is where all executable instructions are stored, and is readable (function pointers) but not writable.

84

POSIX - Process



85

```

1 #include <stdlib.h>
2 #include <stdio.h>
3
4 static int i_stat = 4; /* In data segment */
5 int i_glob; /* In bss segment */
6 int *pi_pg; /* In bss segment */
7
8 /* main in text segment */
9 int main(int nargs, char **args)
10 {
11     /* nargs and args stored in frame 1 of the stack */
12     int *pi_loc; /* stored in frame 1 of the stack */
13     if (!(pi_loc = (int *)malloc(sizeof(int) * 16)))
14         /* 16 x sizeof(int) reserved on the heap */
15         return 1;
16     if (!(pi_pg = (int *)malloc(sizeof(int) * 8)))
17     {
18         /* 8 x sizeof(int) reserved on the heap */
19         free(pi_loc);
20         return 2;
21     }
22     printf("address of i_stat = 0x%10x (program zone, data segment)\n", &i_stat);
23     printf("address of i_glob = 0x%10x (program zone, bss segment)\n", &i_glob);
24     printf("address of pi_pg = 0x%10x (program zone, bss segment)\n", &pi_pg);
25     printf("address of main = 0x%10x (program zone, text segment)\n", main);
26     printf("address of nargs = 0x%10x (pile frame 1)\n", &nargs);
27     printf("address of args = 0x%10x (pile frame 1)\n", &args);
28     printf("address of pi_loc = 0x%10x (pile frame 1)\n", &pi_loc);
29     printf("sbrk(0) = 0x%10x (brk)\n", sbrk(0));
30     printf("pi_loc = 0x%10x (heap)\n", pi_loc);
31     printf("pi_pg = 0x%10x (heap)\n", pi_pg);
32     free(pi_pg);
33     free(pi_loc);
34     return 0;
35 }

```

Output example:

address of i_stat = 0x9a70c010 (program zone, data segment)
 address of i_glob = 0x9a70c020 (program zone, bss segment)
 address of pi_pg = 0x9a70c028 (program zone, bss segment)
 address of main = 0x9a50b78a (program zone, text segment)
 address of nargs = 0xfce053dc (pile frame 1)
 address of args = 0xfce053d0 (pile frame 1)
 address of pi_loc = 0xfce053e0 (pile frame 1)
 sbrk(0) = 0x9c1e5000 (brk)
 pi_loc = 0x9c1c4260 (heap)
 pi_pg = 0x9c1c42b0 (heap)

<https://ilay.org/yann/articles/mem/index.en.html>

86

POSIX - Process API Example

- **fork**: the fork system call clones the current process to create a new process, called a child process.
- The child process executes the next line after the fork() as the parent process does.

```

1 pid_t child_id = fork();
2 if (child_id == -1) {
3     perror("fork");
4     exit(EXIT_FAILURE);
5 }
6 if (child_id > 0) {
7     // We have a child! Get their exit code
8     int status;
9     waitpid( child_id, &status, 0 );
10    // code not shown to get exit status from child
11 } else {
12    // In child ... // start calculation exit(123);
13 }

```

87

POSIX - Process API Example

- Fork will return a non-negative integer on success.
- A child will inherit any open file descriptors of the parent. That means if a parent half of the file and forks, the child will start at that offset. A read on the child's end will shift the parent's offset by the same amount. Any other flags are also carried over.
- Pending signals are not inherited. This means that if a parent has a pending signal and creates a child, the child will not receive that signal unless another process signals the child.
- The process will be created with one thread (more on that later. The general consensus is to not create processes and threads at the same time).

88

POSIX - Process API Example

- Since we have copy on write (COW), read-only memory addresses are shared between processes.
- If a program sets up certain regions of memory, they can be shared between processes.
- Signal handlers are inherited but can be changed.
- The process' current working directory (often abbreviated to CWD) is inherited but can be changed. Environment variables are inherited but can be changed.
- To make the child process execute another program, use one of the exec functions after forking. The exec set of functions replaces the process image with that of the specified program. This means that any lines of code after the exec call are replaced with those of the executed program.
- Process commutation time ~ 30 microseconds

89

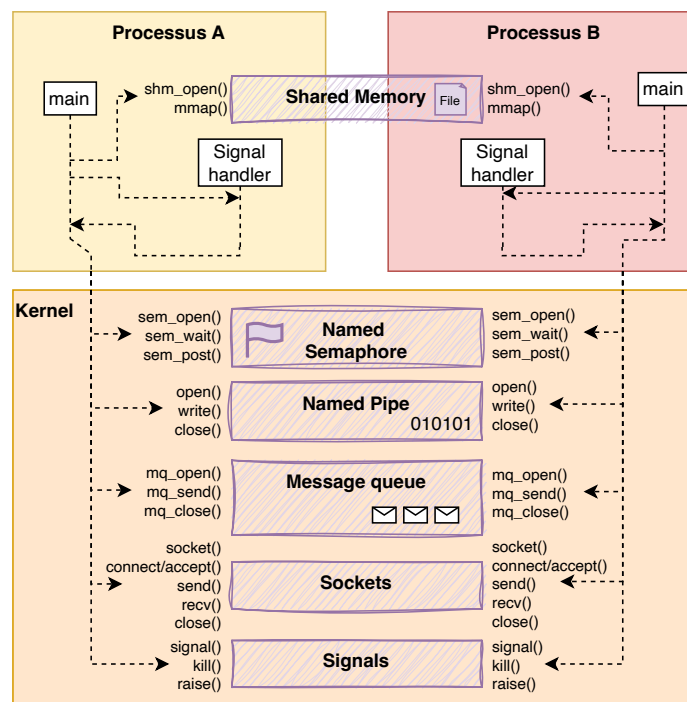
POSIX - Inter-process communication (IPC)

- Message queues allow processes to send formatted data streams to arbitrary processes: **mq_open** (creation), **mq_close** (close without destruction), **mq_unlink** (destruction), **mq_send** (send a message), **mq_receive** (receive a message), **mq_notify** (notify a message is available)
- Shared memory (and shared files) allows processes to share parts of their virtual address space: **shm_open** (creation), **shm_unlink** (destruction), **mmap** (attachment), **munmap**
- Semaphores (named and unnamed) allow processes to synchronize execution: **sem_open**, **sem_close**, **sem_unlink**, **sem_init**, **sem_wait**, **sem_post**,...
- Pipes and Fifos (named and unnamed): **open**, **read**, **write**
- Signals: **signal**, **kill**, **raise**
- Sockets: **socket**, **connect/accept**, **send**, **recv**, **close**

<https://opensource.com/article/19/4/interprocess-communication-linux-storage>

90

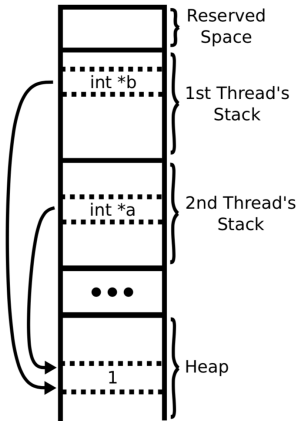
POSIX - Inter-process communication (IPC)



91

POSIX - Thread API Example

- Represents the sequence of instructions that the CPU has and will execute
- Shares the same address space, variables, heap, file descriptors and etc
- To remember how to return from function calls, and to store the values of automatic variables and parameters a thread uses a stack



https://github.com/illinois-cs241/coursebook/blob/pdf_deploy/main.pdf

92

POSIX - Thread API Example

- To use pthreads, include **pthread.h** and compile and link with **-pthread or -lpthread** compiler option
- **pthread_create**: creates a new thread. Every thread gets a new stack.
- **pthread_cancel**: stops a thread. Note the thread may still continue. For example, it can be terminated when the thread makes an operating system call (e.g. write)
- **pthread_exit(void *)**: stops the calling thread meaning the thread never returns after calling pthread_exit
- **pthread_join()**: waits for a thread to finish and records its return value
- In practice, creating threads before forking can lead to unexpected errors because (as demonstrated above) the other threads are immediately terminated when forking
- Thread commutation time ~ 15 microseconds

93

POSIX - Thread API Example

```
1 #include <stdio.h>
2 #include <pthread.h>
3 void *busy(void *ptr) {
4     // ptr will point to "Hi"
5     puts("Hello World");
6     return NULL;
7 }
8 int main() {
9     pthread_t id;
10    pthread_create(&id, NULL, busy, "Hi");
11    void *result;
12    pthread_join(id, &result);
13 }
```

94

POSIX - Mutex API Example

- **Mutex** to ensure that only one thread at a time can access a global variable

```
1 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; // global variable
2 pthread_mutex_lock(&m); // start of Critical Section
3 // Critical section
4 pthread_mutex_unlock(&m); //end of Critical Section
```

- **PCP-like**: if one thread t1 owns the mutex and block thread t2 with a higher priority, t1 inherit of mutex maximum priority allocated

```
1 pthread_mutex_t lock1;
2 pthread_mutexattr_t mutex_attr;
3 pthread_mutexattr_setprotocol(&mutex_attr, PTHREAD_PRIO_PROTECT);
4 pthread_mutexattr_setprioceiling(&mutex_attr, sched_get_priority_max(policy)-1);
5 pthread_mutex_init(&lock1, &mutex_attr);
```

- **PIP-like**: if one thread t1 owns the mutex and block thread t2 with a higher priority, t1 inherit of t2 priority (PTHREAD_PRIO_INHERIT)
- **PCP & PIP are important for determinism**

95

POSIX - Semaphore API Example

- A **semaphore** is another synchronization primitive. It is initialized to some value. Threads can either `sem_wait` or `sem_post` which lowers or increases the value.
- If the value reaches zero and a wait is called, the thread will be blocked until a post is called.

```
1 #include <semaphore.h>
2 sem_t s;
3 int main() {
4     sem_init(&s, 0, 10); // returns -1 (=FAILED) on OS X
5     sem_wait(&s); // Could do this 10 times without blocking
6     sem_post(&s); // Announce that we've finished (and one more resource item is available; increment count)
7     sem_destroy(&s); // release resources of the semaphore
8 }
```

96

POSIX - Conditional Variable API Example

- A **condition variable** is used to suspend a thread until a condition is true, this condition is signal by another thread

```
1 pthread_cond_t cv;
2 pthread_mutex_t m;
3 int count;
4 // Initialize
5 pthread_cond_init(&cv, NULL);
6 pthread_mutex_init(&m, NULL);
7 count = 0;
8 // Thread 1
9 pthread_mutex_lock(&m);
10 while (count < 10) {
11     pthread_cond_wait(&cv, &m);
12     /* Remember that cond_wait unlocks the mutex before blocking
13     (waiting)! */
14     /* After unlocking, other threads can claim the mutex. */
15     /* When this thread is later woken it will */
16
17     /* re-lock the mutex before returning */
18 }
19 pthread_mutex_unlock(&m);
20 //later clean up with pthread_cond_destroy(&cv); and mutex_destroy
```

97

POSIX - Conditional Variable API Example

```
1 // Thread 2:
2 while (1) {
3     pthread_mutex_lock(&m);
4     count++;
5     pthread_cond_signal(&cv);
6     /* Even though the other thread is woken up it cannot not return
7      * from pthread_cond_wait until we have unlocked the mutex. This
8      * is a good thing! In fact, it is usually the best practice to call
9      * cond_signal or cond_broadcast before unlocking the mutex */
10    pthread_mutex_unlock(&m);
11 }
```

98

POSIX - Barrier API Example

- In cases where you must wait for a number of tasks to be completed before an overall task can proceed, **barrier** synchronization can be used.
- The init functions create the barrier, specifying the number of threads that are synchronizing on the barrier
- Threads wait at the barrier until all the threads reach the barrier. When the last thread arrives at the barrier, all the threads resume execution.

```
1 #include <pthread.h>
2 pthread_t tid;
3 pthread_barrier_t barrier;
4 unsigned count;
5
6 void * thread_function(void * arg)
7 {
8     ...
9     pthread_barrier_wait(& barrier);
10    // Your code
11 }
12
13 int main(int argc, char * argv[])
14 {
15     ...
16     count = ...
17     pthread_barrier_init(& barrier, NULL, count);
18     pthread_create(&tid, NULL, thread_function, NULL);
19     ...
20 }
```

99



POSIX - Signal API Example

- **A signal** allows one process to send an event or message to another process. If that process wants to accept the signal, it can, and then, for most signals, decide what to do with that signal.
- In Real-time POSIX implementation: no message lost
- To send a signal: **pthread_sigqueue**
- Waiting a signal: **sigwait, sigwaitinfo, sigtimedwait**
- Signal handler registering: **sigaction**

100

POSIX - Others API

- **Mailbox:** **mq_open, mq_receive, mq_send**
- **Clocks:** **clock_settime, clock_gettime, clock_getres, nanosleep**
- **Timers:** **timer_create, timer_delete, timer_settime, timer_gettime, timer_getoverrun**
- ...

Constants	Description
CLOCK_REALTIME	A settable system-wide real-time clock.
TIMER_ABSTIME	Flag indicating time is absolute with respect to the clock associated with a timer.
CLOCK_MONOTONIC	A nonsettable, monotonically increasing clock that measures time since some unspecified point in the past that does not change after system startup.

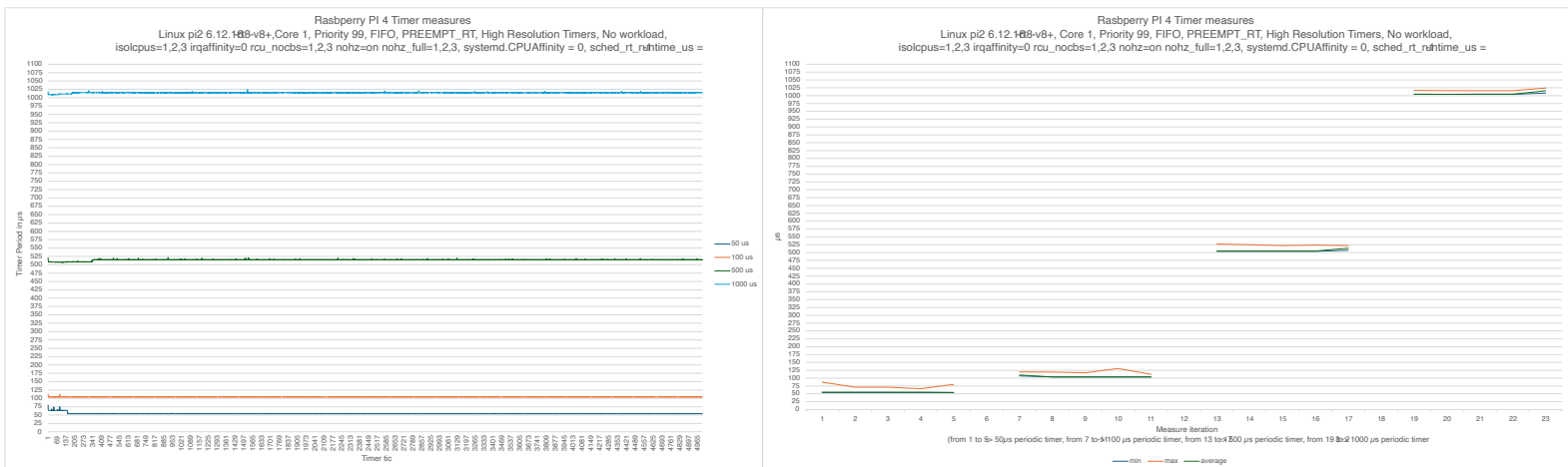
101

POSIX - clock_nanosleep

```
1 ...
2
3 #include <time.h>
4 ...
5
6 #define NS_PER_S 1000000000
7
8 int main(int argc, char *argv[])
9 {
10     struct timespec current_time;
11     struct timespec next_time;
12
13     clock_gettime(CLOCK_MONOTONIC, &current_time);
14     next_time = current_time;
15     next_time.tv_sec += (next_time.tv_nsec + period_ns) / NS_PER_S;
16     next_time.tv_nsec = (next_time.tv_nsec + period_ns) % NS_PER_S;
17     int ret = clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next_time, NULL);
18     ...
19 }
```

102

POSIX - clock_nanosleep



103



Scheduling with POSIX - Temporal Partitioning

- **SCHED_FIFO**: a fixed-priority preemptive scheduling policy, in which processes/threads with the same priority are treated in first-in-first-out (FIFO) order. At least 32 priority levels must be available for this policy.
- **SCHED_RR**: this policy is similar to SCHED_FIFO, but uses a time-sliced (round robin) method to schedule processes/threads with the same priorities. It also has 32 priority
- **SCHED_OTHER**: it is an implementation-defined scheduling policy.

```
1 #include <sched.h>
2 pthread_t tid;
3 pthread_attr_t attr;
4 struct sched_param param;
5 ...
6 pthread_attr_init (&attr);
7 /***** Scheduling policy *****/
8 pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
9 /***** Thread priority *****/
10 param.sched_priority = 5;
11 pthread_attr_setschedparam (&attr, & param);
12 /***** création du thread *****/
13 pthread_create (&tid, &attr, fonc, NULL);
```

104

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Langage generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



105

Integrated Modular Avionics (IMA)

- Traditional federated aircraft controller architecture consists of a large number of different, specialised electronics devices. Each of them is dedicated to a special, singular purpose (e. g. flight control, or fire and smoke detection) and has its own custom sensor/actuator wiring.

Needs:

- Reduction of weight
- Reduction of on-board power consumption
- Lower maintenance costs by reducing the number of different types of replacement units needed to keep on stock
- Reduction of development costs by provision of a standardised operating system, together with standardised drivers for the avionics interfaces most widely used

<https://perso.telecom-paristech.fr/pautet/sar/eter/supports/ima.pdf>

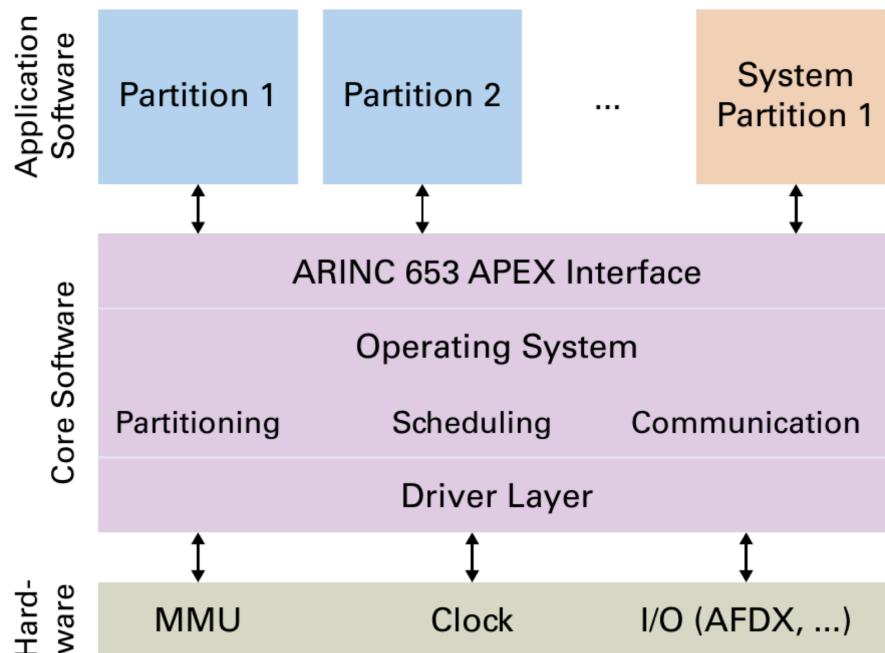
106

Integrated Modular Avionics (IMA)

- **Objective:** To host applications of different safety assurance levels on the same module
- **Impact:** A module must support resource partitioning via memory access protection, strict deterministic scheduling and I/O access permissions
- The interface between avionics applications and the module's operating system conforms to a standardised API which is defined in the ARINC specification 653
- **ARINC-653 (Avionics Application Standard Software Interface)** is a software specification for space and time partitioning in safety-critical avionics real-time operating systems.

107

Integrated Modular Avionics (IMA)



<http://homepages.hs-bremen.de/~jbredereke/downloads/bredereke-tsp-space-avionics-tr-2017.pdf>

108

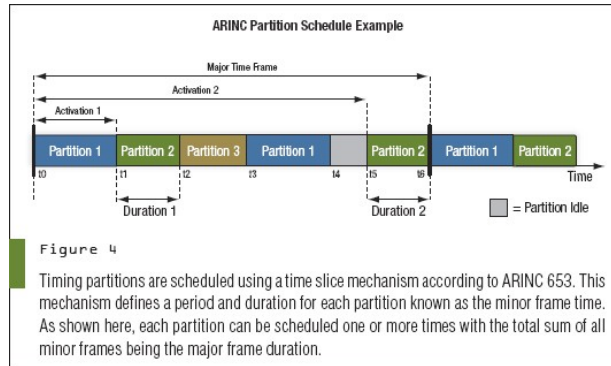
ARINC 653 - APEX

- **API:**
 - Sending and Receiving data to/from other software applications or networked systems
 - Managing resources like time, memory, I/O and displays
 - Handling errors
 - Managing files
 - Scheduling tasks or processes
- **Partition configurations** (All resources needed by a partition are statically allocated in a configuration file.)
 - Memory size
 - Processor utilization (in term of period and duration)
 - Interpartition I/O ports and connections
 - Health monitor actions

109

ARINC 653 - Timing Partitioning

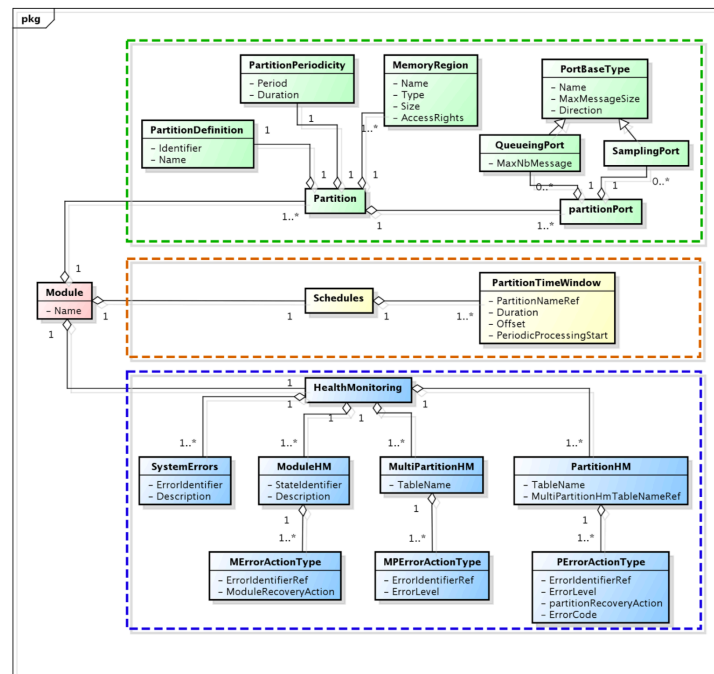
- **Partitions** are scheduled in a static cyclic schedule, which prescribes when and for how long a partition can execute.
- **The major frame** time determines the length of the repeating schedule, and each major frame is divided into partition windows with a specified start time within the major frame and a duration.
- A partition can be scheduled in arbitrarily many partition windows within a major frame.
- Partition schedule **processes** within a partition when its time slot is active.
- Processes within a partition are scheduled according to a priority-based preemptive scheduling scheme (Rate Monotonic Scheduling (RMS) and Earliest Deadline First (EDF))



<https://medium.com/@mehmetcagrikose/a-simple-introduction-to-arinc-653-775b1c5888c0>

110

ARINC 653 - Configuration Data



111

ARINC 653 - Configuration Data

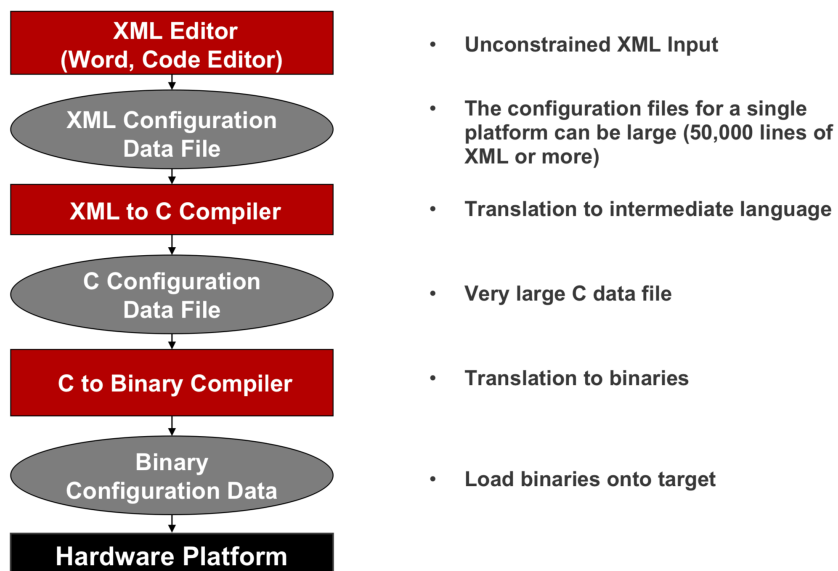
```
<ar:MODULE Name="ARINC 653 Module" xsi:schemaLocation="ARINC653 ModuleExample.xsd"
  xmlns:ar="ARINC653" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <ar:Partitions>
    <ar:Partition>
      <ar:PartitionDefinition Name="systemManagement" Identifier="1"/>
      <ar:PartitionPeriodicity Duration="20000000" Period="20000000"/>
      <ar:MemoryRegions>
        <ar:MemoryRegion Type="RAM" Size="1048576" Name="mainMemory" AccessRights="READ_WRITE"/>
        <ar:MemoryRegion Type="Flash" Size="524288" Name="Flash" AccessRights="READ_ONLY"/>
      </ar:MemoryRegions>
      <ar:PartitionPorts>
        <ar:PartitionPort>
          <ar:QueueingPort MaxMessageSize="30" Name="Stat_2Dq" MaxNbMessage="30" Direction="DESTINATION"/>
        </ar:PartitionPort>
      </ar:PartitionPorts>
    </ar:Partition>
  </ar:Partitions>
  <ar:Schedules>
    <ar:PartitionTimeWindow PeriodicProcessingStart="false" Duration="20000000"
      PartitionNameRef="systemManagement" Offset="0"/>
  </ar:Schedules>
  <ar:HealthMonitoring>
    <ar:SystemErrors>
      <ar:SystemError ErrorIdentifier="5" Description="segmentation error"/>
    </ar:SystemErrors>
    <ar:ModuleHM StateIdentifier="2" Description="system function execution">
      <ar:ErrorAction ErrorIdentifierRef="5" ModuleRecoveryAction="SHUTDOWN"/>
    </ar:ModuleHM>
    <ar:MultiPartitionHM TableName="System partitions HM table">
      <ar:ErrorAction ErrorIdentifierRef="5" ErrorLevel="PARTITION" />
    </ar:MultiPartitionHM>
    <ar:PartitionHM MultiPartitionHMTTableNameRef="System partitions HM table" TableName="systemManagement HM table">
      <ar:ErrorAction ErrorIdentifierRef="5" PartitionRecoveryAction="IDLE"
        ErrorLevel="PROCESS" ErrorCode="MEMORY_VIOLATION" />
    </ar:PartitionHM>
  </ar:HealthMonitoring>
</ar:MODULE>
```

http://www.artist-embedded.org/docs/Events/2007/IMA/Slides/ARTIST2_IMA_WindRiver_Wilson.pdf

112

ARINC 653 - Configuration Data

Typical ARINC 653 XML Compilation



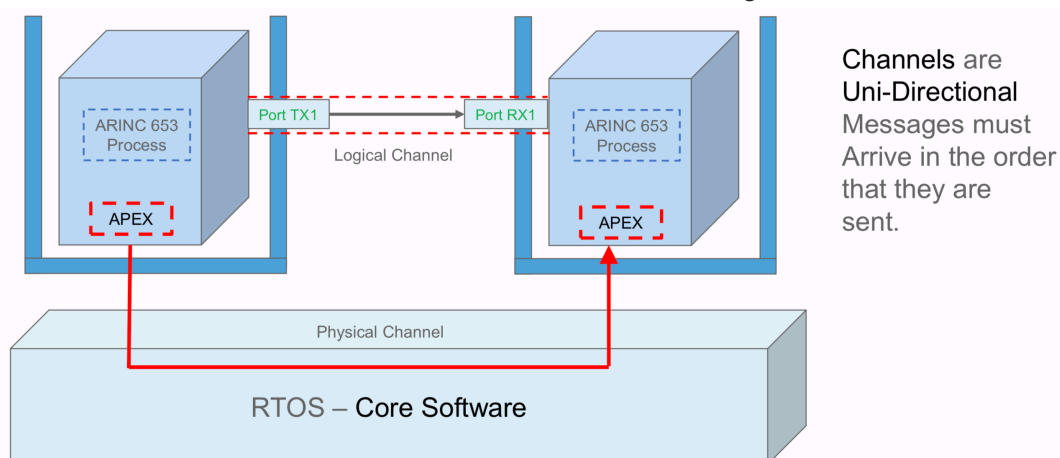
ARINC 653 - Spatial Partitioning

- **Partitions** are contained in a distinct memory segment, which means that each of them has a unique space to store its code and data.
- **Processes** within a partition share the same memory address space
- **Segments properties** (address, size, allocation) are defined at configuration time and cannot be modified during run-time.
- To avoid any access outside their segments, partitions are executed in a user context.
- To use privileged instructions, Partition must call kernel services and performs system calls

114

ARINC 653 - Communication Concepts

- ARINC 653 Communications uses Message Queues or Sampling Ports with the concept of point to point uni-directional channels. Channels, Queues and Ports are defined at design time.

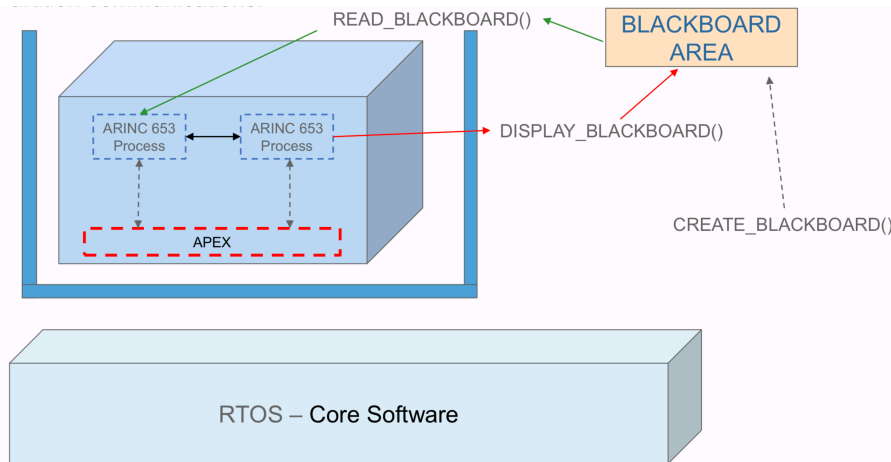


https://www.sysgo.com/fileadmin/user_upload/www.sysgo.com/data/SYSGO_Presentation_ARINC653_RTOS_for_multi-core_certification.pdf

115

ARINC 653 - Communication Concepts

- ARINC 653 Intra Partitions Communications may use Message Queue or Sampling Ports. However, Message Block Buffers and Blockboards may be used instead for intra Partition communications.

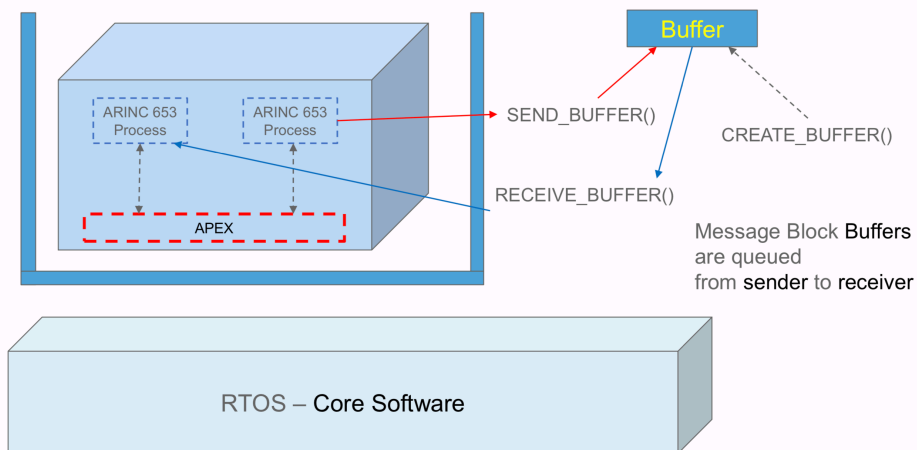


https://www.sysgo.com/fileadmin/user_upload/www.sysgo.com/data/SYSGO_Presentation_ARINC653_RTOS_for_multi-core_certification.pdf

116

ARINC 653 - Communication Concepts

- ARINC 653 Intra Partitions Communications may use Message Queue or Sampling Ports. However, Message Block Buffers and Blockboards may be used instead for intra Partition communications.



https://www.sysgo.com/fileadmin/user_upload/www.sysgo.com/data/SYSGO_Presentation_ARINC653_RTOS_for_multi-core_certification.pdf

117

ARINC 653 - Communication Determinism

- Inter-partition communication
 - For every ARINC port there is a buffer of corresponding size inside the kernel
 - User code while sending to (or receiving from) port accesses those buffers by means of syscalls
 - At the beginning of every major frame arinc kernel copies data from source buffers to destinations. For large buffers there is possibility to spend significant part of partition time slot on buffer-to-buffer copying
 - If a process tries to send to a full port (or read from an empty one) the kernel blocks the process until buffer becomes operational

https://www.ispras.ru/proceedings/docs/2016/28/2/isp_28_2016_2_181.pdf

118

ARINC 653 - Health Monitoring Concepts

- The ARINC 653 Health Monitor defines a number of error levels, supported by Health Monitor configuration Tables and Error Handlers set by Applications.
- Health Monitor is really dealing with actions to be performed on exceptions or system level Errors to protect and recover the system (event levels: Partition, Process, Module, System Error Detection).
- Unhandled events can cause Application halt, Partition Idle, Cold or Warm Boot or even System Halt

https://www.sysgo.com/fileadmin/user_upload/www.sysgo.com/data/SYSGO_Presentation_ARINC653_RTOS_for_multi-core_certification.pdf

119

Basic Comparison of POSIX and ARINC 653 Features

POSIX	ARINC-653 APEX
Event-trigger execution model	Cyclic based execution model to schedule partitions (Time-trigger)
Uses a multiprocessing and multithreading execution model	Same concept as POSIX; however, terminology differs: Uses partitions instead of processes and processes instead of threads
Does not support temporal partitioning in the norm (Although, Kernel implementing the POSIX interface can/should/shall)	Does support temporal partitioning using round-robin scheduling between partitions
Uses priority preemptive scheduling for processes and threads. Cyclic scheduling can be programmed	Supports priority preemptive scheduling for processes within a partition, and cyclic scheduling between partitions
Input/Output scheduling depends on the device drivers and may be interrupt-driven or polled	Many models are possible, with interrupt-driven while the partition doing the I/O is running or polled I/O to get better control (i.e. I/O on request)
Supports memory segregation	Same concepts as POSIX; however terminology is different: Uses spatial partitioning instead of memory segregation. Memory layout, access policies, cache policies and memory protection are provided using static configuration tables
Uses sockets	Uses sampling and queing ports on I/O
Uses interprocess communication	Communicates using sampling and queing ports or through shared memory controlled by the memory management system
Mutex and condition variables	Buffer, blackboard, semaphore, event
Uses timers	API services can respond to timeouts (wait). Processes can be periodic; delay mechanisms and other mechanisms can be used to synchronize application code with time
Uses Signal management and requires user to establish error response policies	Uses health monitoring to manage user policies established in configuration tables or ties in to used-defined error handlers

Developing Safety-Critical Software: A Practical Guide for Aviation Software - Chapter 20 - Leanna Rierson - 2017

120

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Langage generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



121

Focus on Design Concepts in API descriptions

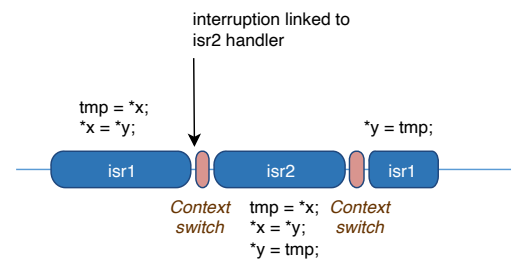
- **Threadsafe:** An operation is “thread-safe” if it can be performed from multiple threads safely, even if the calls happen simultaneously on multiple threads. A threadsafe function protects shared resources from concurrent access by locks.
- **Reentrant:** Reentrant code is one that can be executed before an earlier/simultaneous execution has finished, without affecting the control and data flow of the first execution. An operation is re-entrant if it can be performed simultaneously in either the same or different contexts. This is a stronger concept than threadsafe, because the operations can occur within the same thread. A reentrant function does not hold static data over successive calls, nor does it return a pointer to static data.

122

Focus on Design Concepts in API descriptions

Exercise: Reentrant ? Threadsafe ?

```
1 int tmp;
2
3 void swap(int* x, int* y)
4 {
5     tmp = *x;
6     *x = *y;
7     /* Hardware interrupt might invoke isr2() here. */
8     *y = tmp;
9 }
10
11 void isr1()
12 {
13     int x = 1, y = 2;
14     swap(&x, &y);
15 }
16
17 void isr2()
18 {
19     int x = 3, y = 5;
20     swap(&x, &y);
21 }
```



123

Focus on Design Concepts in API descriptions

- **Race condition:** This happens when the result of multiple thread execution in critical section differs according to the order in which the threads execute. The outcome of a program is determined by its sequence of events determined by the processor. This means that the execution of the code is non-deterministic. Meaning that the same program can run multiple times and depending on how the kernel schedules the threads could produce inaccurate results.

<https://linux.developpez.com/actu/279534/Google-decouvre-des-centaines-de-situations-de-competition-dans-le-noyau-Linux-en-se-servant-de-KCSAN-son-nouveau-detecteur-de-courses-critiques-pour-le-noyau-Linux/>

124

Thread and Race Conditions Example (1/2)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/syscall.h>
5
6 void* myfunc(void* ptr) {
7     int i = *((int *) ptr);
8     printf("Thread %ld known by the kernel as %i receives: %i\n", pthread_self(), syscall(SYS_gettid), i);
9     return NULL;
10 }
11
12 int main() {
13     // Each thread gets a different value of i to process
14     int i;
15     pthread_t tid[10];
16     for(i=0; i < 10; i++)
17         pthread_create(&tid[i], NULL, myfunc, &i);
18     for(int i=0; i<10; i++)
19         pthread_join(tid[i], NULL);
20 }
```

125

Thread and Race Condition Example (2/2)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/syscall.h>
5
6 void* myfunc(void* ptr) {
7     int i = ((int) ptr);
8     printf("Thread %ld known by the kernel as %i receives: %i\n", pthread_self(), syscall(SYS_gettid), i);
9     return NULL;
10 }
11
12 int main() {
13     // Each thread gets a different value of i to process
14     int i;
15     pthread_t tid[10];
16     for(i=0; i < 10; i++)
17         pthread_create(&tid[i], NULL, myfunc, (void *)i);
18     for(int i=0; i<10; i++)
19         pthread_join(tid[i], NULL);
20 }
```

126

Exercise - Process & Thread API Example (1/3)

```
1 #include <sys/types.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <sys/wait.h>
6 #include <pthread.h>
7
8 int cptr;
9
10 void *produce(void* params)
11 {
12     int i = *((int*)params);
13     sleep(i);
14     if(cptr % 2)
15         cptr = cptr - i;
16     else
17         cptr = cptr + i;
18     printf("Thread %i produce cptr = %i\n", i, cptr);
19     pthread_exit(NULL);
20 }
```

127

Exercise - Process & Thread API Example (2/3)

```
1 int main() {
2     pid_t pid;
3     int i,status;
4     pthread_t tid [10];
5
6     if ((pid = fork()) < 0)
7         perror("fork() error");
8     else if (pid == 0) {
9         for(i = 0;i< 10;i++)
10            pthread_create(&tid[i],NULL,produce,&i);
11    }
12    else {
13        if (wait(&status) == -1){
14            perror("wait() error");
15        }else if (WIFEXITED(status)){
16            /* WIFEXITED returns true if the child process exited normally. In this case
17             WEXITSTATUS(status) returns the exit status of the child process: the return
18             value of the child process.
19             */
20            printf("The child exited with status of %d\n", WEXITSTATUS(status));
21        }else{
22            printf("The child did not exit successfully");
23        }
24        printf("CPTR: %i\n",cptr);
25    }
26    exit(EXIT_SUCCESS);
27 }
```

128

Exercise - Process & Thread API Example (3/3)

- How many processes and threads in the previous program ?
- This program compiles and executes with the following result:

```
1 The child exited with status of 0
2 CPTR: 0
```

- Whereas the expected result is:

```
1 Thread 0 produce cptr = 0
2 Thread ... /*One print for each thread with the expected identifier*/
3 The child exited with status of 9
4 CPTR: 9
```

- Why ?

129



Focus on Design Concepts in Kernels: Lock-free & Wait-free

- **Lock-free data structure:** safe for concurrent access without locks. Some thread makes progress with every step (enable maximum concurrency & robustness).
- **Wait-free data structure:** a lock-free data structure with the additional property that every thread accessing the data structure can complete its operation within a bounded number of steps, regardless of the behavior of other threads. Every thread can make forward progress, regardless of what the other threads are doing; there's no need for waiting.
- Lock-free programming has the following advantages:
 - Can be used in places where locks must be avoided, such as interrupt handlers
 - Efficiency benefits compared to lock-based algorithms for some workloads, including potential scalability benefits on multiprocessor machines
 - Avoidance of priority inversion in real-time systems
- Lock-free programming requires the use of special atomic processor instructions, such as CAS (compare and swap), LL/SC (load linked/store conditional), or the C Standard atomic_compare_exchange generic functions.

130

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Language generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



131

Focus on C language: C11 Atomic types

- C11 (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>) defines a new **__Atomic** type specifier:

```
1 "#include<stdatomic.h>"
2 __Atomic int b;
```

- Such qualified object guarantees that any read or write access to it is atomic
- The type_name can not be an array type, a function type, an atomic type, or a *qualified type* (https://en.wikipedia.org/wiki/Type_qualifier).
- We could define a use-defined struct atomic types, which enable the struct to be manipulated atomically.
- The **__Atomic** keyword makes it possible to use objects of all atomic types in nearly all the same expressions and contexts as their direct: initialize like a normal int type

```
1 __Atomic int b = 3;
2 ...
3 b = b + 2; //is an atomic addition
4 ...
```

<https://hal.inria.fr/hal-01236734/document>

132

Focus on C language: C11 Atomic types

- Guarantees that between different threads all standard operations (defined through operators such as += or functional such as atomic_exchange) are perceived as indivisible
- **WARNING:** Only given between threads and in perception: in reality an operation can well be divided into several processor instructions, a read-modify-write operations such as a simple i++ may be split into several CPU instructions, and the perception guarantee does not extend to visibility between the main program and signal handlers (An operation that extends perception of indivisibility to signal handlers is called lock-free in the jargon of the C standard)
- **Needs hardware and compiler support** for the individual atomic operations and a generic atomic lock interface in the C library when no low-level atomic instruction is available

<https://hal.inria.fr/hal-01236734/document>

133

Focus on C language: C11 Atomic types

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdatomic.h>
4
5 _Atomic int acnt;
6 int cnt;
7 void *adding(void *input)
8 {
9     for(int i=0; i<10000; i++)
10     {
11         acnt++;
12         cnt++;
13     }
14     pthread_exit(NULL);
15 }
16 int main()
17 {
18     pthread_t tid[10];
19     for(int i=0; i<10; i++)
20         pthread_create(&tid[i], NULL, adding, NULL);
21     for(int i=0; i<10; i++)
22         pthread_join(tid[i], NULL);
23
24     printf("the value of acnt is %d\n", acnt);
25     printf("the value of cnt is %d\n", cnt);
26     return 0;
27 }
```

```
gcc -std=c11 -pthread -o test test.c
./test
the value of acnt is 100000 (correct)
the value of cnt is 89824 (wrong)
```

134

Focus on C language: C11 Atomic APIs

```
1 atomic_flag_test_and_set //sets an atomic_flag to true and returns the old value
2 atomic_flag_clear        //sets an atomic_flag to false
3 atomic_init              //initializes an existing atomic object
4 atomic_is_lock_free      //indicates whether the atomic object is lock-free
5 atomic_store             //stores a value in an atomic object
6 atomic_load              //reads a value from an atomic object
7 atomic_exchange          //swaps a value with the value of an atomic object
8 atomic_fetch_add         //atomic addition
9 atomic_fetch_sub         //atomic subtraction
10 atomic_compare_exchange_strong //swaps a value with the an atomic object if the old value is what is expected, otherwise reads the old value
11 atomic_compare_exchange_weak  //swaps a value with the an atomic object if the old value is what is expected, otherwise reads the old value
12 ...
```

```
1 _Bool atomic_compare_exchange_weak(volatile A *object, C *expected, C desired);
```

- Description: Atomically, compares the value pointed to by object for equality with that in expected, and if true, replaces the value pointed to by object with desired, and if false, updates the value in expected with the value pointed to by object. (Here, A means atomic type, while the C means the non-atomic type corresponding to A, for example, if A is `_Atomic int`, then, C is `int`)
- Test and set approach
- Usefull for lock-free and wait-free algorithms

135

Focus on C language: C11 Atomic C11 APIs and Lock-Free algorithm

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdatomic.h>
4
5 _Atomic int lock = 0;
6 int lock_count = 0;
7 int unlock_count = 0;
8
9 void *counting(void *input)
10 {
11     int expected = 0;
12     for(int i = 0; i < 100000; i++)
13     {
14         unlock_count++;
15         while(!atomic_compare_exchange_weak(&lock,&expected,1))
16             //if the lock is 0(unlock), then set it to 1(lock).
17             expected = 0;
18         //if the CAS fails, the expected will be set to 1,
19         //so we need to change it to 0 again.
20         lock_count++;
21         lock = 0;
22     }
23     pthread_exit(NULL);
24 }
25
26 int main()
27 {
28     pthread_t tid[10];
29     for(int i=0; i<10; i++)
30     {
31         pthread_create(&tid[i],NULL,counting,NULL);
32     }
33     for(int i=0; i<10; i++)
34         pthread_join(tid[i],NULL);
35
36     printf("the value of lock_count is %d\n",lock_count);
37     printf("the value of unlock_count is %d\n",unlock_count);
38 }
```

gcc -std=c11 -pthread -o test test.c

./test

the value of lock_count is 1000000 (correct)

the value of unlock_count is 772024 (wrong)

136

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Language generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



137

Clean code

1. Memory Partitioning and Temporal Partitioning are managed by the Operating System/Kernel through partitions **but do not design an algorithm with hypothesis on the scheduling or execution order.**
2. **Acquire and release synchronization primitives in the same module, at the same level of abstraction.**
3. **Avoid using locks in interruption handlers and if required use dedicated kernel ones.**
4. **Do not use volatile as a synchronization primitive:**
 - The volatile keyword informs the compiler that the qualified variable may change in ways that cannot be determined; consequently, compiler optimizations must be restricted for memory areas marked as volatile. For example, the compiler is forbidden to load the value into a register and subsequently reuse the loaded value rather than accessing memory directly. This concept relates to multithreading because incorrect caching of a shared variable may interfere with the propagation of modified values between threads, causing some threads to view stale data.
5. **Ensure visibility when accessing shared variables.** To ensure the visibility of the most recent update, **the write to the variable must happen before the read** (C Standard, subclause 5.1.2.4, paragraph 18 [ISO/IEC 9899:2011]). Atomic operations—other than relaxed atomic operations—trivially satisfy the happens before relationship. Where atomic operations are inappropriate, protecting both reads and writes with a mutex also satisfies the happens before relationship.

<https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=87151970>

138

Clean code

6. **Join or detach threads even if their exit status is unimportant.** Whenever a thread terminates without detaching, the thread's stack is deallocated, but some other resources, including the thread ID and exit status, are left until it is destroyed by either join or detach calls. These resources can be vital for systems with limited resources and can lead to various "resource unavailable" errors.
7. **Do not perform operations that can block while holding a lock.**
8. **Ensure that compound operations on shared variables are atomic.** Compound operations are operations that consist of more than one discrete operation. Expressions that include postfix or prefix increment (++), postfix or prefix decrement (--), or compound assignment operators always result in compound operations. Compound assignment expressions use operators such as *=, /=, %=, +=, -=, <<=, >>=, ^=, and |=. Compound operations on shared variables must be performed atomically to prevent data races.
9. **Do not assume that a group of calls to independently atomic methods is atomic.** A consistent locking policy guarantees that multiple threads cannot simultaneously access or modify shared data. When two or more operations must be performed as a single atomic operation, a consistent locking policy must be implemented using some form of locking, such as a mutex. In the absence of such a policy, the code is susceptible to race conditions.

<https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=87151970>

139

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Language generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



140

Focus on Spatial Partitioning

- To guarantee **space isolation**, kernel often relies on:
 - Static link customization offline with a **Memory Protection Unit (MPU)** usage online
 - Virtual memory address space with a **Memory Management Unit (MMU)** from which different protection mechanisms can be used (segmentation, pagination, etc.)
 - A POSIX Process thinks that it has a long list of contiguous memory addresses starting at 0
 - A POSIX Process really has a series of small, mostly contiguous chunks of memory all over the physical memory space of the computer
- When switching from a partition to another, the kernel ensures that:
 - Execution context from the previous partition (e.g. POSIX Process) is **clean** (the elected partition cannot read data produced by the previous).
 - The memory segments/pages associated with the elected partition can be **loaded**.
 - **Caches are flushed** so that the execution context is clean as if the partition was freshly elected on a clear environment. (Cache lock systems exist between cores of some processors
<http://events19.linuxfoundation.org/wp-content/uploads/2017/11/Introducing-Cache-Pseudo-Locking-to-Reduce-Memory-Access-Latency-Reinette-Chatre-Intel.pdf>)

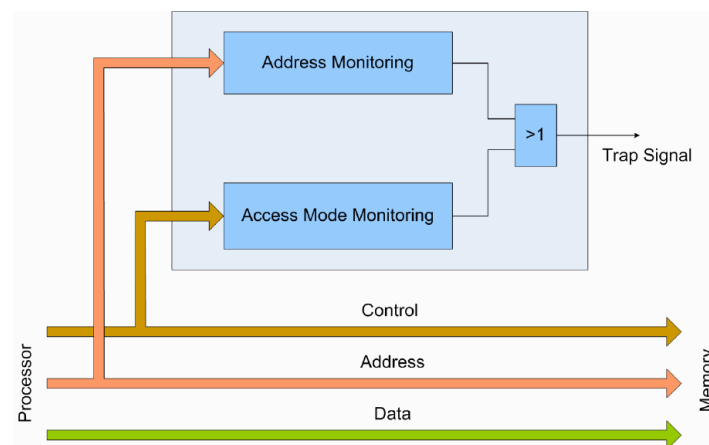
141

Focus on Spatial Partitioning with MPU

- The hardware MPU module **performs a continuous monitoring** of the CPU address bus: the contents of the CPU address bus are compared with the valid address range, either by specifying the beginning and the ending of the allowed addressing space or through the specification of base address and address space size parameters.
- In addition to the monitoring of the addressing range, the **memory access mode can also be monitored**. For example, a given address space can be configured for: **read access only; read and write access; read and execute; execute only; etc.**
- Memory protection methods based on address monitoring are **simple to implement**.
- Possible fine-grained range of address protection (e.g. double word natural boundary in 32-bit processors);
- An inherent low susceptibility to bit-flips caused by single-event upsets.

142

Focus on Spatial Partitioning with MPU



<http://air.di.fc.ul.pt/air/downloads/07-35.pdf>

143



Focus on Spatial Partitioning with MMU

- The hardware MMU module performs a **translation of the valid addressing space**: a given logical address is translated into a (linear) physical address, usually from a base address specified through a Memory Management Unit (MMU) descriptor.
- **Each memory access is checked** by the processor Memory Management Unit (MMU), using memory and/or I/O addressing descriptors tables. A trap signal is generated upon the detection of an unauthorized access.
- It allows the mapping of different logical addresses in the same physical address references thus creating some sort of address alias (in the different partitions for the same communication channel for example)
- Segmentation/Paging allows a program's virtual memory to be **non-contiguous in physical memory**
- It potentially exhibits a higher vulnerability to bit-flips caused by single-event upsets.

144

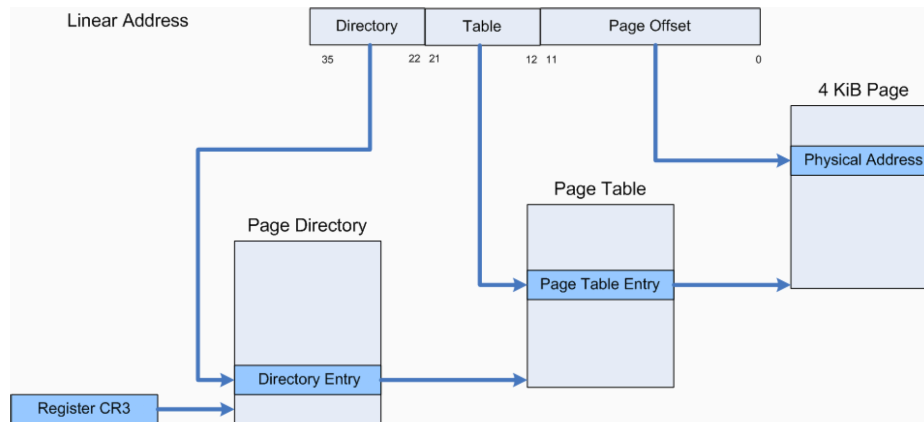
Focus on Spatial Partitioning with MMU - Paging

- **Partition address space is divided into fixed-size blocks** of a particular size, e.g. 4KB, 8KB for both virtual and physical memory.
- Blocks of physical memory are called frames, blocks of virtual memory are called pages.
- **Kernel is responsible to allocate pages.**
- Every virtual page is mapped to a corresponding frame in physical memory through page tables.
- Paging technique is faster in terms of memory access.
- Pages can specify additional characteristics: Read-only vs. read-write, executable code vs. data.

<https://moodle.insa-lyon.fr/pluginfile.php/16715/course/section/1476/5-demand-paging.pdf>

145

Focus on Spatial Partitioning with MMU - Paging



Intel IA-32 Page-based Memory Protection Mechanisms <http://air.di.fc.ul.pt/air/downloads/07-35.pdf>

146

POSIX and Memory Partitioning - Just for fun !

```

1 #include <unistd.h>
2 #include <signal.h>
3 #include <stdio.h>
4 #include <malloc.h>
5 #include <stdlib.h>
6 #include <errno.h>
7 #include <sys/mman.h>
8
9 int main(int argc, char *argv[])
10 {
11     char *p;
12     int pagesize;
13     struct sigaction sa;
14
15     sa.sa_flags = SA_SIGINFO;
16     sigemptyset(&sa.sa_mask);
17     sa.sa_sigaction = handler;
18     if (sigaction(SIGSEGV, &sa, NULL) == -1)
19         handle_error("sigaction");
20
21     pagesize = sysconf(_SC_PAGE_SIZE);
22     if (pagesize == -1)
23         handle_error("sysconf");
24
25     /* Alloue un tampon aligné sur une frontière de page ;
26        la protection initiale est PROT_READ | PROT_WRITE */
27     buffer = memalign(pagesize, 4 * pagesize);
28     if (buffer == NULL)
29         handle_error("memalign");
30     printf("Début de la région : 0x%lx\n", (long) buffer);
31
32     if (mprotect(buffer + pagesize * 2, pagesize,
33                 PROT_NONE) == -1)
34         handle_error("mprotect");
35
36     for (p = buffer ; ; )
37         *(p++) = 'a';
38     printf("Boucle terminée\n"); /* Ne devrait jamais arriver */
39     exit(EXIT_SUCCESS);
40 }

```

```

1 #define handle_error(msg) \
2     do { perror(msg); exit(EXIT_FAILURE); } while (0)
3
4 char *buffer;
5
6 static void handler(int sig, siginfo_t *si, void *unused)
7 {
8     printf("Reçu SIGSEGV à l'adresse : 0x%lx\n",
9           (long) si->si_addr);
10    exit(EXIT_FAILURE);
11 }

```

Output example:

```

$ ./a.out
Début de la région : 0x804c000
Reçu SIGSEGV à l'adresse : 0x804e000

```

147

Focus on Temporal Partitioning - schedulers

- **Temporal partitioning** among different partitions is guaranteed since at the end of a partition time slot, the kernel **preempts all internal threads, disables all interrupt sources belonging to this partition and finally sets up the environment for the next partition** (context switch).
- Exemple of the partition scheduling in ARINC-653 and threads scheduled within those partition intervals
- Example of the Complete Fair Linux Scheduler that preempts tasks at the end of the timeslot:
<https://developer.ibm.com/tutorials/l-completely-fair-scheduler/>

148

Summary

1. Generalities
2. POSIX interface
3. ARINC 653 approach
4. API design generalities
5. C Langage generalities for concurrency
6. Clean code rules
7. Spatial and temporal partitioning implementations
8. Linux specific attention points



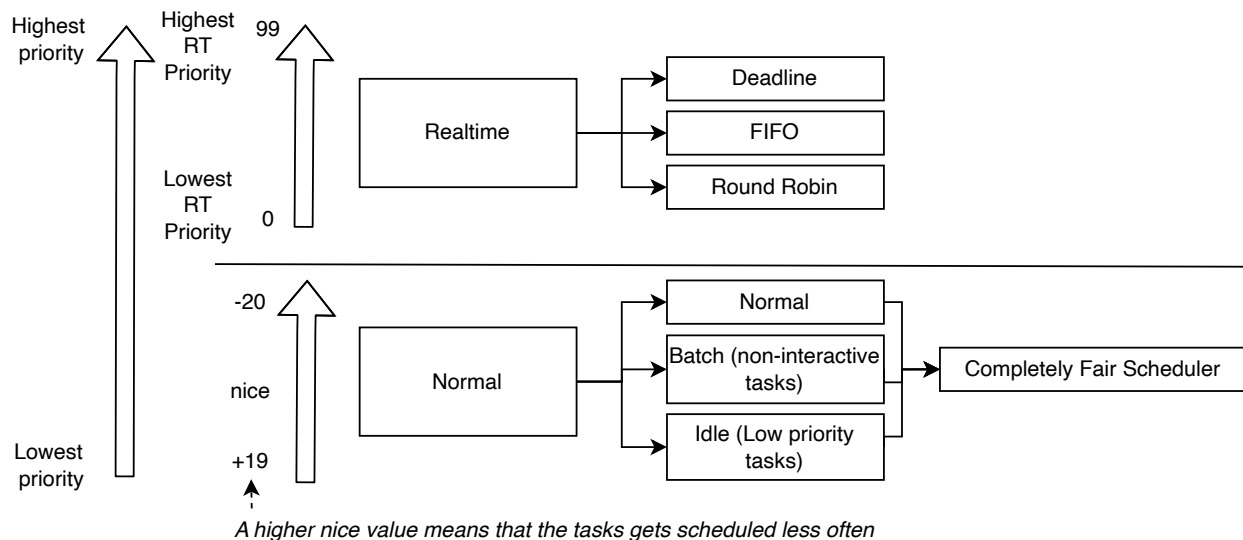
149

Scheduling with Linux - Temporal Partitioning (1/2)

- Linux supports temporal partitioning with several scheduling policies based on the following properties
 - Priority based
 - Preemptible execution (except kernel)
 - Dynamic task sequencing (scalability)
 - SMP & best-effort design
- POSIX SCHED_OTHER/SCHED Normal**: Normal policies (Default policy, with lowest priority) with by default the usage of the Completely Fair Scheduler
- POSIX SCHED_FIFO/SCHED_RR**: Realtime policies (Highest priority than Normal tasks) with specific schedulers for each sub-policy
- SCHED_DEADLINE** => Since version 3.14, Linux provides a deadline scheduling policy implemented using a Global Earliest Deadline First policy

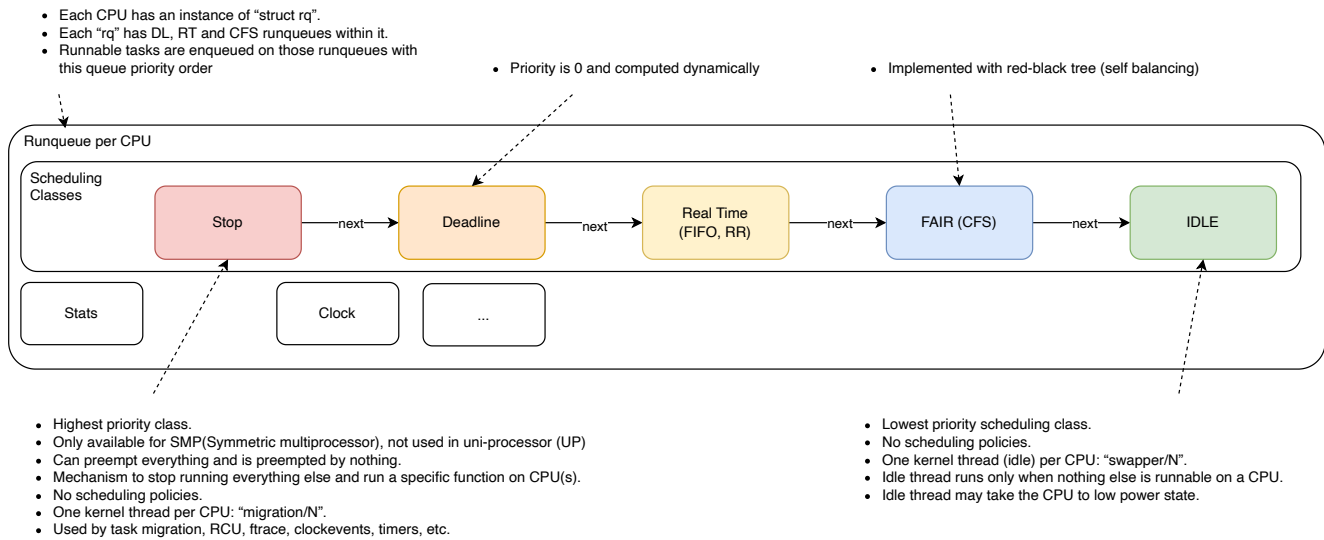
150

Scheduling with Linux - Temporal Partitioning (2/2)



151

Scheduling with Linux - Temporal Partitioning



<https://static.linaro.org/connect/yvr18/presentations/yvr18-220.pdf> <https://helix979.github.io/jkoo/post/os-scheduler/> https://eci.intel.com/docs/3.3/development/performance/rt_scheduling.html#chrt-runtime-processes-linux-scheduling-policies <https://deepdives.medium.com/digging-into-linux-scheduler-47a32ad5a0a8>

152

Focus on Temporal Partitioning - Core management (1/8)

- It is possible to allocate a process to a specific core through:

- The taskset shell command

```
1 chrt -f 99 ./myprog
2 -f: SCHED_FIFO
3 -r: SCHED_RR
4 -d: SCHED_DEADLINE
```

- The `int sched_setaffinity` system call (next slides)

- It is also possible to set an affinity for the threads of a process (not a POSIX function)

Function	Description
<code>CPU_ZERO(cpu_set_t *set)</code>	initializes set to be empty
<code>CPU_SET(int cpu, cpu_set_t *set)</code>	adds the CPU <code>cpu</code> to set
<code>CPU_CLR(int cpu, cpu_set_t *set)</code>	removes the CPU <code>cpu</code> to set
<code>CPU_ISSET(int cpu, cpu_set_t *set)</code>	returns true if the CPU <code>cpu</code> is a member of set
<code>int sched_setaffinity(pid_t pid, size_t len, cpu_set_t* cpuset)</code>	sets the CPU affinity of the process specified by <code>pid</code>
<code>int sched_getaffinity(pid_t pid, size_t len, cpu_set_t* cpuset)</code>	retrieves the CPU affinity mask of the process specified by <code>pid</code>
<code>int pthread_setaffinity_np(pid_t pid, size_t len, cpu_set_t* cpuset)</code>	sets the CPU affinity of the process specified by <code>pid</code> (np suffix is used for non-portable function)
<code>int pthread_getaffinity_np(pid_t pid, size_t len, cpu_set_t* cpuset)</code>	retrieves the CPU affinity mask of the process specified by <code>pid</code> (np suffix is used for non-portable function).

153

Focus on Temporal Partitioning - Core management (2/8)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sched.h>
5
6 int main(int argc, char *argv[])
7 {
8     cpu_set_t cpuset;
9     const int cpu = 0;
10
11     CPU_ZERO(&cpuset);
12     CPU_SET(cpu, &cpuset);
13
14     if (sched_setaffinity(0, sizeof(cpuset), &cpuset) != 0) {
15         perror(argv[1]);
16         exit(EXIT_FAILURE);
17     }
18
19     while (1) {
20         fprintf(stdout, "I'm on CPU #%d\n", sched_getcpu());
21         fflush(stdout);
22         sleep(1);
23     }
24     return EXIT_SUCCESS;
25 }
```

154

Focus on Temporal Partitioning - Core management (3/8)

```
1 ...
2 #include <pthread.h>
3
4 void * entrypoint(void * arg)
5 {
6     //Thread behavior
7 }
8
9 int main(void)
10 {
11     cpu_set_t cpuset;
12     pthread_t thr;
13     pthread_attr_t attr;
14     long num = 0;
15
16     while (1) {
17         pthread_attr_init(& attr);
18         CPU_ZERO(&cpuset);
19         CPU_SET(num, &cpuset);
20         pthread_attr_setaffinity_np(&attr,
21                                     sizeof(cpu_set_t),
22                                     &cpuset);
23
24         if (pthread_create(&thr, &attr, entrypoint, (void *) num) != 0)
25             break;
26         num++;
27     }
28     pthread_exit(NULL);
29 }
```

155

Focus on Temporal Partitioning - Core management (4/8)

- Jitter can affect tasks assigned by the symmetric multiprocessing (SMP) balancing and scheduling algorithms. You can isolate CPUs, so tasks won't be assigned to them by those algorithms.
- It is possible to isolate a cpu: The **isolcpus** defines a set of CPUs where the kernel processes scheduler will not interact with those CPUs:
 - Syntax: **isolcpus=1,2,3** //keep core 0 for the scheduler
 - Without intervention, the process scheduler will not put newly created processes on a CPU listed in **isolcpus**.
 - Without intervention, the process scheduler will not migrate processes onto or off of any CPU listed in **isolcpus**. This includes migrating processes from/to regular CPUs to/from CPUs listed in **isolcpus** and migrating processes between CPUs listed in **isolcpus**.
 - A process can be moved onto an **isolcpus** CPU by a systems administrator or script with tools such as **taskset**.
 - A process can also put itself onto a CPU listed in **isolcpus** with scheduling-specific functions such as **sched_setaffinity()**
 - For either Cgroups or systemd, the process scheduler will still automatically migrate processes around in the Cgroup CPU set and/or the list of CPUs designated by CPUAffinity. For **isolcpus** the process scheduler will not touch processes in the **isolcpus** list. As such, if a process is on an **isolcpus** CPU, it will stay on that CPU until it finishes executing or yields the CPU.

156

Focus on Temporal Partitioning - Core management (5/8)

- The default CPU that handles interrupts is the **CPU 0** but the SMP algorithms may assign any CPU to service any IRQ.
- Protecting a CPU from having to service IRQs can remove significant jitter from its remaining tasks.
- To restrict which CPUs are available for servicing interrupts, use the **irqaffinity=0** boot cmdline parameter
- IRQs can be also pinned to CPUs by tweaking **/proc/irq/XX/smp_affinity**
- The **irqbalance** tool monitors and distributes the irq affinity to spread the load across CPUs
- Use the **IRQBALANCE_BANNED_CPUS** environment variable to make **irqbalance** ignore some CPUs

157

Focus on Temporal Partitioning - Core management (6/8)

- read-copy-update (RCU) achieves scalability improvements by allowing reads to occur concurrently with updates.
- In mainline Linux, RCU carries out significant processing in softirq context, during which preemption is disabled.
- These callbacks often free memory, and the memory allocators can therefore impose large latencies when they take their slowpaths.
- Although these latencies cannot be avoided, they can be directed to the CPUs of your choice through use of RCU callback offloading. Callbacks for Linux's read-copy-update mechanism are assigned to candidate CPUs. By removing a CPU from the candidate list you can reduce jitter in its remaining tasks.
- When RCU callbacks are offloaded from a CPU, it has more opportunity to enter dyntick-idle or adaptive-tick mode. That helps the NO_HZ facility to reduce scheduler jitter.
- To allow that, compile the kernel with the CONFIG_RCU_NOCB_CPU=y and to offload RCU callbacks from specified CPUs, use the kernel command-line parameter: `rcu_nocbs=1,2,3`

https://wiki.linuxfoundation.org/realtime/documentation/technical_details/rcu#:~:text=To%20enable%20callback%20offloading%20on,1%2C%203%2C%20and%204.

158

Focus on Temporal Partitioning - Core management (7/8)

- It is useful to keep the **systemd** services separated from the real-time application. You can do this by setting the CPUAffinity parameter in the `/etc/systemd/system.conf` file to the cores you want to isolate.
- Deferred execution mechanism like Workqueues can be pinned to CPUs in `/sys/devices/virtual/workqueue/cpumask`
- The `kthread_cpus` parameter refers to the list of CPUs specifically allocated for kernel usage. For instance, specifying `kthread_cpus=0-3` means that the kernel is allowed to run kernel threads on CPUs 0 through 3.

159

Scheduling with Linux - EDF & Core Management (8/8)

- WARNING: EDF scheduling over multiprocessor is a NP scheduling problem. To keep the kernel "simple", core exploration is not supported by the scheduler (sched_setaffinity is used by the scheduler)
- You have to resolve the allocation before through CGroup cpuset

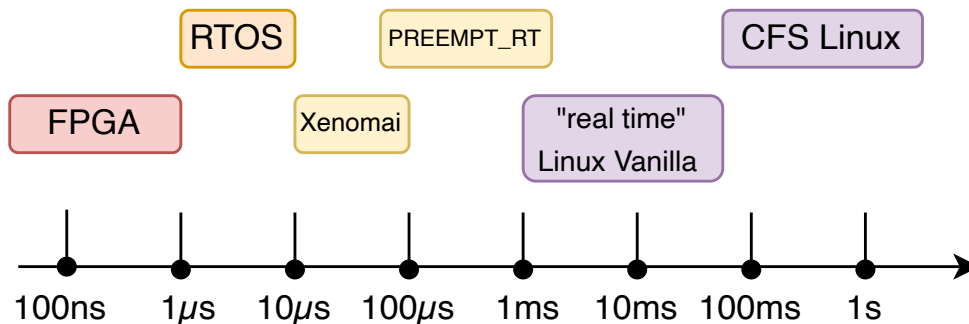
```
1 mkdir /dev/cpuset
2 mount -t cgroup -o cpuset cpuset /dev/cpuset
3 cd /dev/cpuset
4 mkdir cpu0
5 echo 0 > cpu0/cpuset.cpus
6 echo 0 > cpu0/cpuset.mems
7 echo 1 > cpuset.cpu_exclusive
8 echo 0 > cpuset.sched_load_balance
9 echo 1 > cpu0/cpuset.cpu_exclusive
10 echo 1 > cpu0/cpuset.mem_exclusive
11 echo $$ > cpu0/tasks
12 // Next executions are allocated to the cpu0 group (core 0)
```

<https://stackoverflow.com/questions/50165719/sched-setaffinity-for-sched-deadline>

160

Focus on Temporal Partitioning - Timers (1/7)

- Linux kernel allows to measure the time with an accuracy of the order of one microsecond
- Specific compilation flags High Resolution Timers and Tickless System allows software timers with an accuracy of the order of tens or hundreds of microseconds
- **Linux is not a real-time operating system**



161

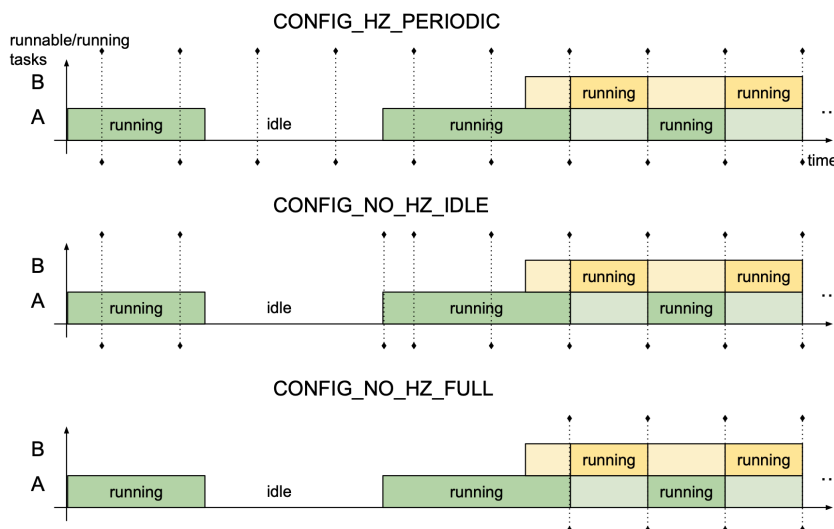
Focus on Temporal Partitioning - Timers (2/7)

- The Scheduler is invoked on a regular basis to perform time-sharing activities
- This is sequenced through the system ticks, generated by a high resolution timer
- Several policies regarding system ticks are available:
 - **CONFIG_HZ_PERIODIC**: Always tick at a given rate. This introduces small interferences but is deterministic.
 - **CONFIG_NO_HZ_IDLE**: Disable the tick when idle, for powersaving interferences but is deterministic.
 - **CONFIG_HZ_PERIODIC**: Always tick at a given rate. Longer wakeup from Idle, replay the missed ticks (jiffies housekeeping)
 - **CONFIG_NO_HZ_FULL**: Actively disables ticking even when not idle.
 - This **NO_HZ** facility can reduce jitter, by vetoing unnecessary scheduling-clock interrupts
 - Per-CPU setting, through the **nohz=on nohz_full=1,2,3** boot parameter
 - Only relevant on multi-core systems, one core must stay in nohz_idle mode
 - The list must not include the CPU that boots the system

<https://bootlin.com/doc/training/preempt-rt/preempt-rt-slides.pdf>

162

Focus on Temporal Partitioning - Timers (3/7)



<https://bootlin.com/doc/training/preempt-rt/preempt-rt-slides.pdf>

163

Focus on Temporal Partitioning - Timers (4/7)

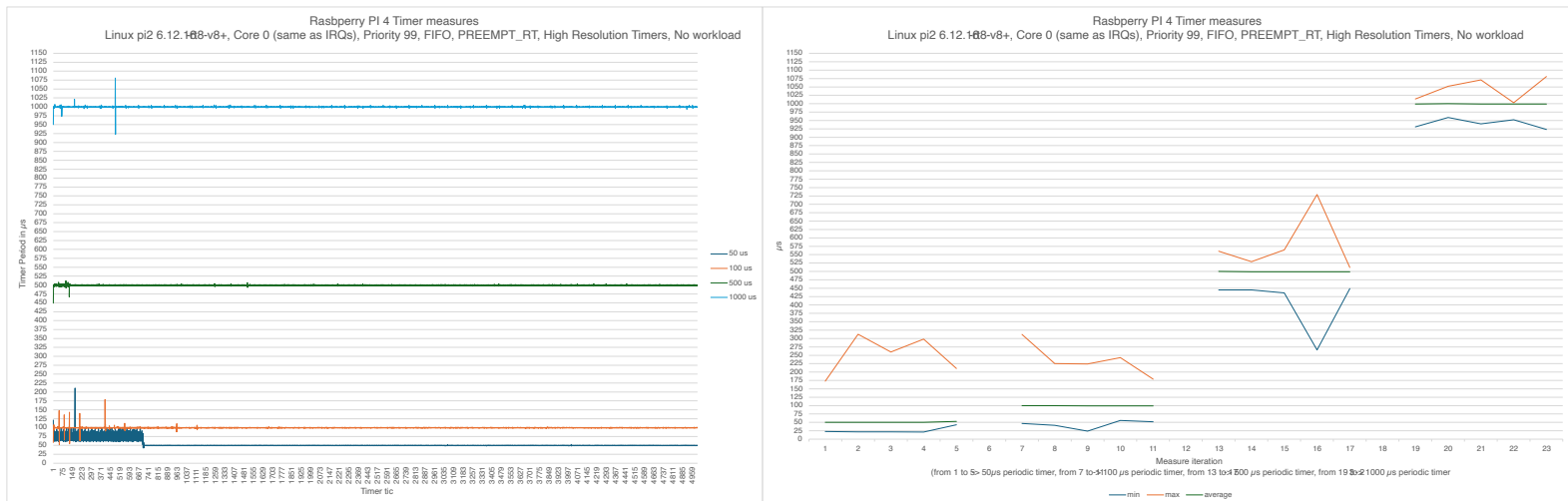
```

1 ...
2
3 #include <time.h>
4 ...
5 void handler_signal(int arg)
6 {
7     ...
8 }
9
10 int main(int argc, char * argv[])
11 {
12     ...
13     // Timer configuration
14     signal(SIGRTMIN, handler_signal);
15     event.sigev_notify = SIGEV_SIGNAL;
16     event.sigev_signo = SIGRTMIN;
17     period = period * 1000; // in nanosec
18     spec.it_interval.tv_sec = period / 1000000000;
19     spec.it_interval.tv_nsec = period % 1000000000;
20     spec.it_value = spec.it_interval;
21     // Timer allocation
22     if (timer_create(CLOCK_REALTIME, &event, &timer) != 0) {
23         perror("timer_create");
24         exit(EXIT_FAILURE);
25     }
26     if (timer_settime(timer, 0, &spec, NULL) != 0) {
27         perror("timer_settime");
28         exit(EXIT_FAILURE);
29     }
30     ...

```

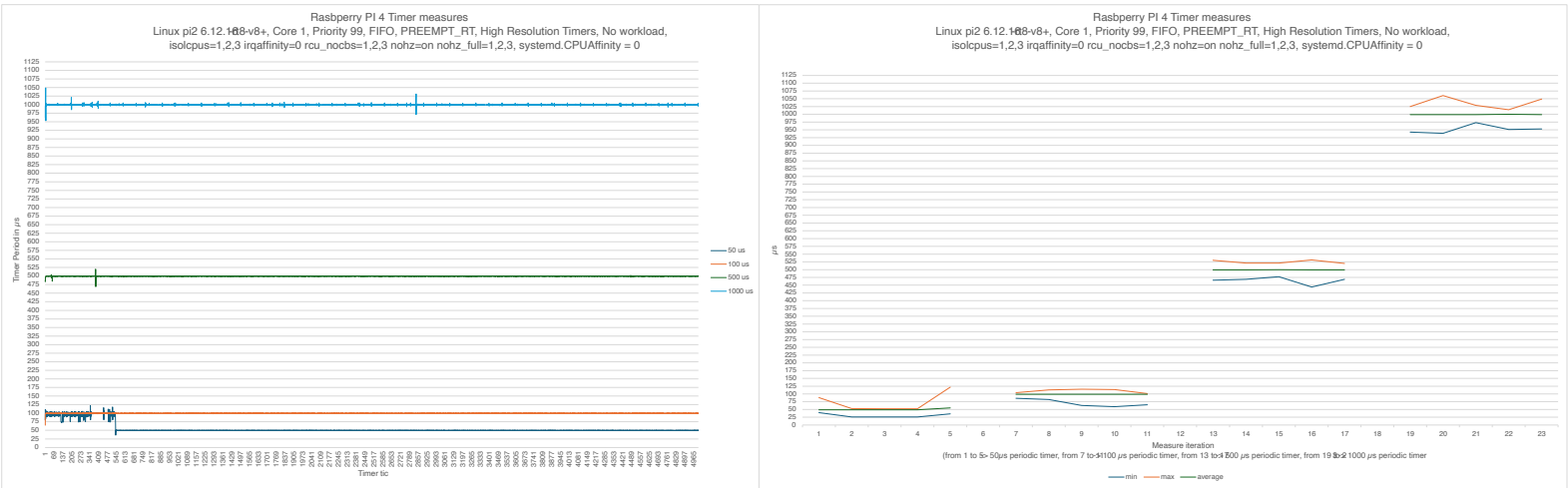
164

Focus on Temporal Partitioning - Timers - Linux is not a real-time operating system (5/7)

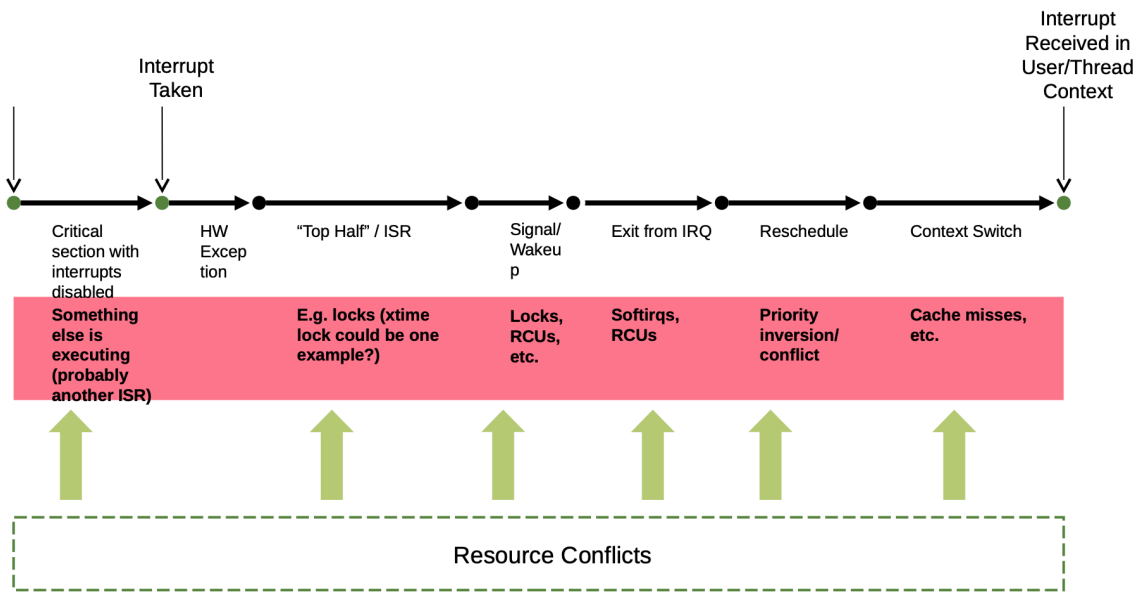


165

Focus on Temporal Partitioning - Timers - Isolation Configuration(6/7)



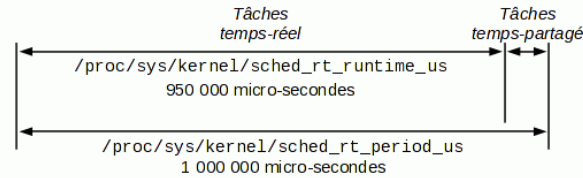
Focus on Temporal Partitioning - Timers - Linux Latency (7/7)



<https://wiki.csie.ncku.edu.tw/embedded/rt-linux-smp.pdf>

Focus on Temporal Partitioning - Real-time Safeguard (1/2)

- In Linux, a safeguard mechanism exists that allows the system administrator to allocate bandwidth for use by real-time tasks: **WARNING can break the determinism**

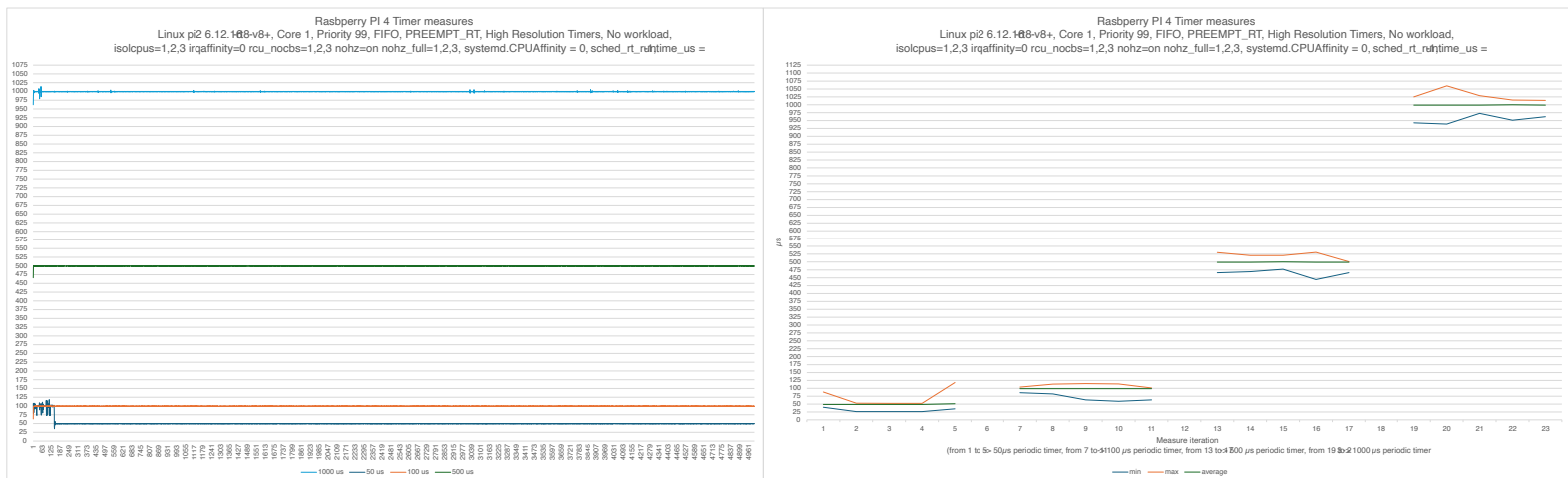


- `/proc/sys/kernel/sched_rt_period_us` defines the period in μs (microseconds) to be considered as 100% of CPU bandwidth. The default value is 1,000,000 μs (1 second)
- `/proc/sys/kernel/sched_rt_runtime_us` defines the total bandwidth available to all real-time tasks.
- The default values is 950,000 μs (0.95 s) or, in other words, 95% of the CPU bandwidth. Setting the value to -1 means that real-time tasks may use up to 100% of CPU times.

```
1 echo -1 > /proc/sys/kernel/sched_rt_runtime_us
2 #or
3 sysctl kernel.sched_rt_runtime_us=-1
```

168

Focus on Temporal Partitioning - Real-time Safeguard (2/2)



169

Focus on Temporal Partitioning - Preemption (1/4)

```
1 int main(int argc, char * argv[])
2 {
3     //time threshold of 25 us
4     long int threshold = 25;
5     struct timeval start;
6     struct timeval now;
7     struct timeval previous = {0, 0};
8     long long int duration;
9
10    gettimeofday(&start, NULL);
11    previous = start;
12    while(1) {
13        gettimeofday(&now, NULL);
14        duration = now.tv_sec - previous.tv_sec;
15        duration *= 1000000;
16        duration += now.tv_usec - previous.tv_usec;
17        previous = now;
18        if (duration >= threshold)
19            fprintf(stderr, "%lld\n", duration);
20        // Pause each five seconds
21        if (now.tv_sec - start.tv_sec > 5) {
22            usleep(100000);
23            gettimeofday(&start, NULL);
24            previous = start;
25        }
26    }
27    return EXIT_SUCCESS;
28 }
```

- This program is allocated to core 0 with the highest priority 99, threshold is arbitrary set to 25 us

Extract from chapter 8 of <https://www.blaess.fr/christophe/livres/solutions-temps-reel-sous-linux/>

170

Focus on Temporal Partitioning - Preemption (2/4)

```
1 $ taskset -pc 0 $$
2 $ sudo chrt -f 99 ./exemple-interruption-01
3 35
4 124
5 129
6 [...]
7     $ ping 192.168.3.104
8     PING 192.168.3.104 (192.168.3.104) 56(84) bytes of data.
9 98
10 58
11     64 bytes from 192.168.3.104: icmp_seq=1 ttl=64 time=0.450 ms
12 86
13 43
14     64 bytes from 192.168.3.104: icmp_seq=2 ttl=64 time=0.373 ms
15 43
16 41
17     64 bytes from 192.168.3.104: icmp_seq=3 ttl=64 time=0.356 ms
18 53
19 40
20     64 bytes from 192.168.3.104: icmp_seq=4 ttl=64 time=0.332 ms
21 44
22 29
23     64 bytes from 192.168.3.104: icmp_seq=5 ttl=64 time=0.300 ms
24     (Contrôle-C)
25 $
26 (Contrôle C)
```

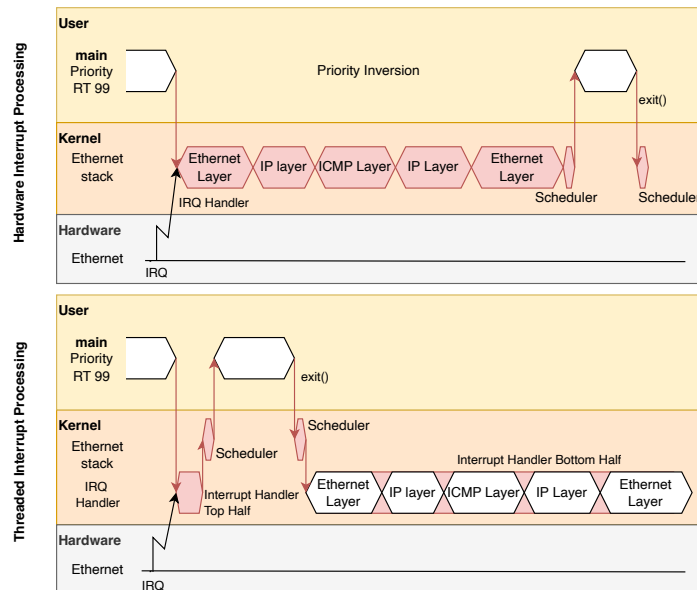
Priority Inversion with the Ethernet Stack for classical linux, two solutions:

- PREEMPT_RT patch
- A dual kernel system: RTAI, Xenomai

Extract from chapter 8 of <https://www.blaess.fr/christophe/livres/solutions-temps-reel-sous-linux/>

171

Focus on Temporal Partitioning - Preemption (3/4)



172

Focus on Temporal Partitioning - Preemption (4/4)

- Interrupt handlers run with interrupts disabled
- In PREEMPT_RT, almost all interrupt handlers are threaded
- Very small hardware interrupt handlers are used, that have a well-defined execution time
- They acknowledge the interrupts, and enqueues the "real" interrupt handler
- The interrupt handler runs in a dedicated Kernel thread
- Threaded interrupts are well established in the mainline kernel
- Change kernel spinlocks to "preemptive spinlocks". With PREEMPT_RT, spinlock_t and rwlock_t will become sleeping locks.
- Kernel preemption: CONFIG_PREEMPT_RT - Fully Preemptible Kernel (Real-Time)
- Other added functions to add determinism in the kernel: Tracing, Printk...

<https://bootlin.com/doc/training/preempt-rt/preempt-rt-slides.pdf>

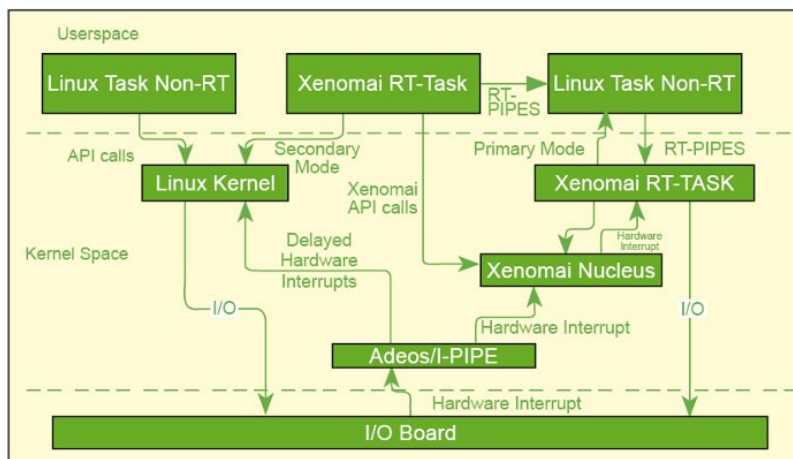
173

Focus on Temporal Partitioning - Dual Architecture

- Xenomai Kernel and Linux Kernel work side by side without one kernel being aware that the other kernel is working in parallel: the Xenomai kernel (the primary mode) for real-time tasks, the Linux kernel (the secondary mode) for other tasks
- Both can communicate with each other via a virtual interrupt controller or I-pipe.
- Switching between modes takes place automatically: WARNING about the syscalls to the linux kernel in real-time tasks...Need specific drivers

174

Focus on Temporal Partitioning - Dual Architecture



<https://www.opensourceforu.com/2015/10/the-xenomai-project-a-linux-based-rtos/>

175

Focus on Temporal Partitioning - Page fault management (1/3)

- Linux Virtual Memory management used for spatial partitioning implies page fault exception management.
- The main reasons for page faulting are:

Exception	Type	Action
Region valid but page not allocated	Minor	Allocate a page frame from the physical page allocator
Region not valid but is beside an expandable region like the stack	Minor	Expand the region and allocate a page
Page swapped out but present in swap cache. The latter case never happens on real time systems as swap space is deactivated and system memory is controlled to always be low.	Minor	Re-establish the page in the process page tables and drop a reference to the swap cache. The latter case never happens on real time systems as swap space is deactivated and system memory is controlled to always be low.
Page swapped out to backing storage	Major	Find where the page with information stored in the PTE and read it from disk
Page write when marked read-only	Minor	If the page is a copy-on-write page, make a copy of it, mark it writable and assign it to the process. If it is in fact a bad write, send a SIGSEGV signal
Region is invalid or process has no permissions to access	Error	Send a SEGSEGV signal to the process
Fault occurred in the kernel portion address space	Minor	If the fault occurred in the vmalloc area of the address space, the current process page tables are updated against the master page table held by init_mm. This is the only valid kernel page fault that may occur
Fault occurred in the userspace region while in kernel mode	Error	If a fault occurs, it means a kernel system did not copy from userspace properly and caused a page fault. This is a kernel bug which is treated quite severely.

- Each page fault will raise an exception, preempt the process and introduce latency**
- fttrace tool can be used to trace pagefaults

```
1 $ trace-cmd record -e page_fault* ./myprog
```

176

Focus on Temporal Partitioning - Page fault management (1/3)

- This dynamic behavior used for spatial partitioning changes the execution time of a program.
- The Linux kernel always tries to provide memory which is not really physically reserved (principle of overcommit - copy on write for example)
- In order to prevent page fault, we have to make sure three conditions are met :
 - All memory allocation must be made during initialization
 - A page fault is triggered for each allocated page
 - No page is given back to the kernel and the kernel does not swap it
- The programmer must take care of the first condition during implementation. The two last ones can be managed by mlockall()

177

Focus on Temporal Partitioning - Page fault management (2/3)

- The `mlockall` subroutine causes all of the pages mapped by the address space of a process to be memory-resident until unlocked or until the process exits or executes another process image.

Function	Description
<code>mlock</code>	Locks a specified region of a process's address space
<code>mlockall</code>	Locks all of a process's address space
<code>munlock</code>	Unlocks a specified region of a process's address space
<code>munlockall</code>	Unlocks all of a process's address space

- The flags parameter `MCL_CURRENT` | `MCL_FUTURE` determines whether the pages to be locked are those currently mapped by the address space of the process, those that are mapped in the future, or both.
- The caller is not privileged, but needs privilege (`CAP_IPC_LOCK`) to perform the requested operation.

```

1 ...
2 #define MY_STACK_SIZE 32768
3
4 void lock_memory() {
5     unsigned char table[MY_STACK_SIZE];
6     memset(table,0,MY_STACK_SIZE);
7     if(mlockall(MCL_CURRENT | MCL_FUTURE) !=0) {
8         perror("mlockall()");
9         exit(EXIT_FAILURE);
10    }
11 }
```

<https://www.blaess.fr/christophe/livres/solutions-temps-reel-sous-linux/>

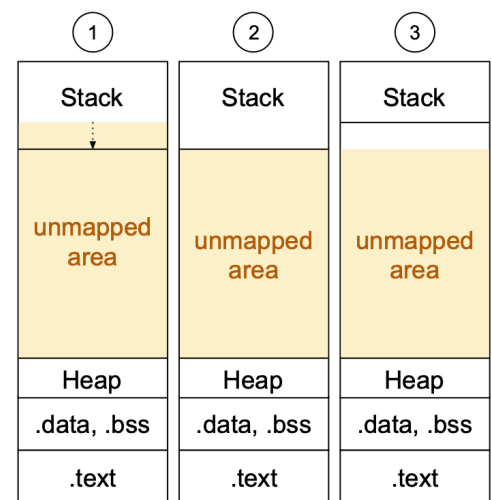
178

Focus on Temporal Partitioning - Pre-faulting the stack

- For each Thread:** allocate huge buffers on the stack in a sub-function
- Access (read or write) the whole buffer
- return from the sub-function

```

1 ...
2 #define BUFSZ (1024 * 1024 * 8)
3 static void stack_prefault() {
4     volatile char buff[BUFSZ];
5     long page_sz;
6     int i;
7     page_sz = sysconf(_SC_PAGESIZE);
8     for (i = 0; i < BUFSZ; i += page_sz)
9         buff[i] = 0;
10 }
```



<https://bootlin.com/doc/training/preempt-rt/>

179

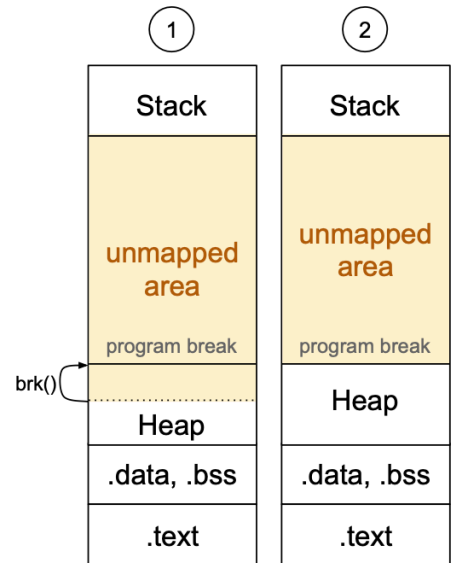
Focus on Temporal Partitioning - Pre-faulting the heap

- The program break (brk) defines the size of the .data segment
- malloc uses the .data segment as a pool for small buffers
- Prevent the program break from shrinking: `mallopt` controls the algorithm used by `malloc`.

```
mallopt(M_TRIM_THRESHOLD, -1);
```

- Prevent malloc from using mmap
 - By default, malloc uses mmap for large buffers
 - Once free'd, mmap'd pages won't be re-used
 - Setting `M_MMAP_MAX` to 0 disables the use of mmap for servicing large allocation requests

```
mallopt(M_MMAP_MAX, 0);
```

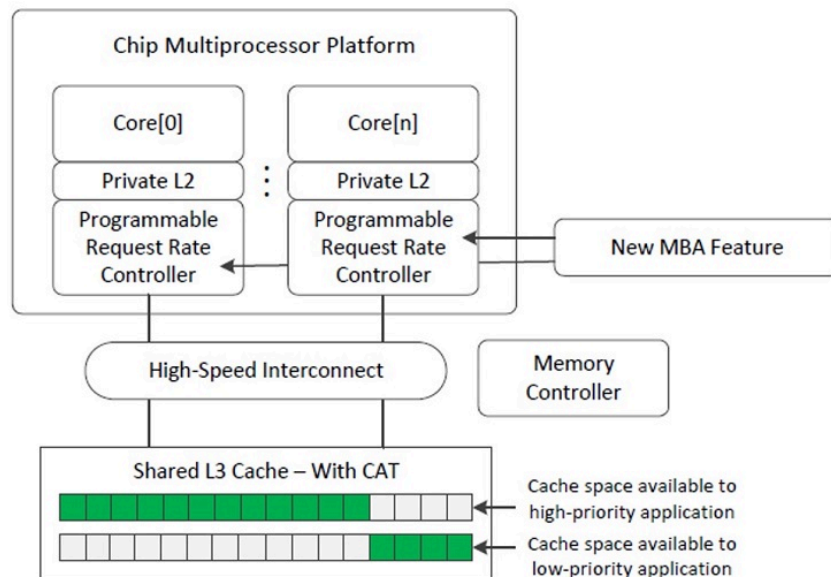


1. Allocate a huge buffer to move the program break
2. Access the whole buffer
3. `free()` the buffer
4. Subsequent calls to `malloc` will re-use pre-faulted memory

<https://bootlin.com/doc/training/preempt-rt/>

180

Focus on Temporal Partitioning - Cache management



<https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-memory-bandwidth-allocation.html>

<https://github.com/intel/intel-cmt-cat/tree/master>

181

Focus on Temporal Partitioning - Cache management

- The Intel Cache Allocation Technology (CAT) architecture defines a per-software-thread tag called a Class of Service (CLOS or COS), which enables running threads, applications or VMs to be mapped to a particular bandwidth setting in the underlying hardware

```
1 #Associate cores 0, 2, and 6 to 10 with COS 1 and core 1 to COS 2:
2 > pqos -a "llc:1=0,2,6-10;llc:2=1;"
```

- In typical usages an enabled Operating System (OS), hypervisor or Virtual Machine Monitor (VMM) will maintain the association of threads to CLOS. Typically when a software thread is swapped onto a given logical processor, the Intel IA32_PQR_ASSOC MSR will be updated to reflect the CLOS of the thread, which allows hardware to select the correct memory bandwidth limit to apply.

<https://github.com/intel/intel-cmt-cat/wiki/Usage-Examples>

<https://www.intel.com/content/www/us/en/developer/articles/technical/cache-allocation-technology-usage-models.html>

182

Focus on Temporal Partitioning - Cache management

- In typical usages an enabled Operating System (OS), hypervisor or Virtual Machine Monitor (VMM) will maintain the association of threads to CLOS. Typically when a software thread is swapped onto a given logical processor, the Intel IA32_PQR_ASSOC model-specific registers will be updated to reflect the CLOS of the thread, which allows hardware to select the correct memory bandwidth limit to apply.
- Configuration of the L3 capacity bitmasks per logical class of service (CLOS), via the IA32_L3_MASK_n block of model-specific registers

Cache Allocation Technology (CAT) Example - 20 bit Mask																			
	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	0
CLOS[0]: Mask	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
CLOS[1]: Mask	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
CLOS[2]: Mask	0	0	0	0	0	0	0	0	1	1	1	1	1	1	0	0	0	0	0
CLOS[3]: Mask	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1

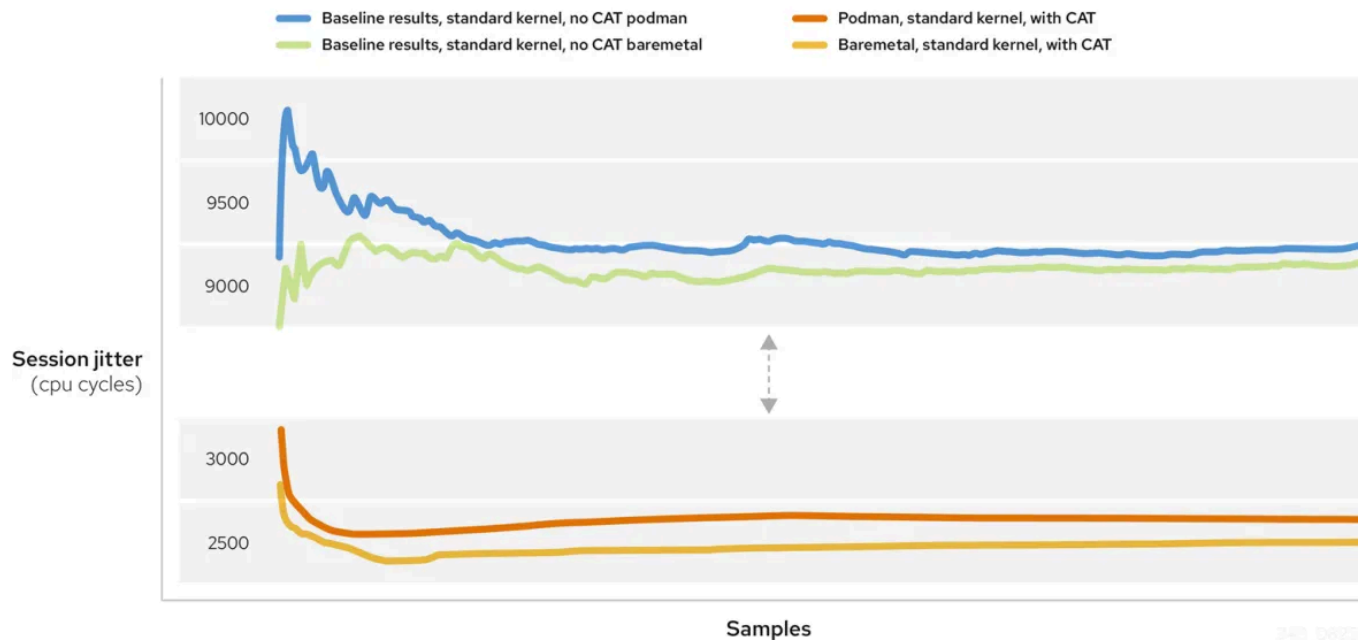
<https://github.com/intel/intel-cmt-cat/wiki/Usage-Examples>

<https://www.intel.com/content/www/us/en/developer/articles/technical/cache-allocation-technology-usage-models.html>

183

Focus on Temporal Partitioning - Cache management

- Example of a Redhat experimentation with an 11th Gen Intel(R) Core(TM) i5-1145GRE @ 2.60GHz and RAM 16GB DDR4 3200MHz (podman for container technology).



<https://www.redhat.com/en/blog/performance-tuning-at-the-edge>

184

Focus on Temporal Partitioning - CPU idle management

- Configure the hardware to limit idle states at runtime or at boot-time
- WARNING: The CPU frequency can also be dynamically changed through Dynamic Voltage and Frequency Scaling

185

How to measure/profile/trace multithreaded programs on Linux, e.g. lock contentions and performances?

- ValgrinD DRD <https://valgrind.org/docs/manual/drd-manual.html>
- **Systemtap** <https://sourceware.org/systemtap/>
- Bcc
 - <https://github.com/iovisor/bcc>
 - https://jugurthab.github.io/debug_linux_kernel/linux-tracers-utility.html
- Linux Tracing workshop <https://github.com/goldshtn/linux-tracing-workshop>
- **eBPF** <https://isovalent.com/books/learning-ebpf/> and <https://ebpf.io/labs/>
- Les secrets du traceur eBPF - <https://www.linuxembedded.fr/2019/03/les-secrets-du-traceur-ebpf/>
- **rt-tests** packages which is a collection of programs for testing the real-time capabilities of a Linux system. **cyclicttest** is a program to measure maximum latency ([cyclicttest in action - https://documentation.ubuntu.com/real-time/en/latest/how-to/measure-maximum-latency/#](https://documentation.ubuntu.com/real-time/en/latest/how-to/measure-maximum-latency/#))
- Several tools explained in the chapter Testing and Benchmarking of <https://bootlin.com/doc/training/preempt-rt/preempt-rt-slides.pdf>
- Embedded probes...